TUTORIAL 2 • TRANSFORMER ARCHITECTURE • LINE BY LINE

Attention, worked out from scratch

The complete transformer notebook, explained — **every line of code, every formula derived**, every output and figure exactly as it ran, and full worked solutions to all twelve exercises. Written for someone who is new to Python and wants the mathematics spelled out rather than asserted.

BASED ON	CODE
Tutorial_2_Transformer_Architecture_Hands_On.ipynb	**NumPy only — no framework, no GPU**
OUTPUTS	**COVERS**
Executed, not transcribed	**6 core cells + 12 exercises**

---

00 Attention is a soft dictionary lookup

Before any code: if you hold one mental image for this whole notebook, hold this one.

An ordinary Python dictionary does a **hard** lookup. You supply a key, it matches exactly one stored key, and you get back exactly one value:

- `d["animal"]` → matches the key `"animal"` → returns its value.
- Miss by one character and you get nothing at all.

Attention does the same thing **softly**. Every token puts out a **query** (“what am I looking for?”), every token advertises a **key** (“here is what I am”), and every token offers a **value** (“here is what I would contribute”). Instead of one key winning, *every* key gets a matching score, those scores are squashed into weights that sum to 1, and the answer is a weighted blend of all the values.

THE WHOLE FORMULA

That is all the famous equation says. Read it right to left as “compare, normalise, blend”:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$ core

QK^T compares every query with every key. $1/\sqrt{d_k}$ stops those comparisons from growing with model width. **softmax** turns them into weights that sum to 1. Multiplying by V blends the values using those weights.

Why “soft” is the entire trick

A hard lookup is not differentiable — nudge the query slightly and either nothing changes or the answer jumps to a different entry. There is no gradient to learn from. A soft lookup changes smoothly: nudge the query and the weights shift a little, so calculus works, so the model can be trained. Every design decision in this notebook follows from wanting a lookup you can differentiate.

What you will build

PIECE	WHAT IT DOES	SECTION
Scaled dot-product attention	The core lookup, in five lines of NumPy	02–03
A worked linguistic example	Watch “it” find its antecedent	06–07
Multi-head attention	Several lookups in parallel, then concatenate	09
Positional encoding	Give the model a sense of word order	11
A full encoder block	Attention → add & norm → FFN → add & norm	13
Twelve exercises	Including causal masking and stacking blocks	04–18

No deep-learning framework, no GPU, no API key — only NumPy and matplotlib.
Everything here is arithmetic you could in principle do on paper.

01 The NumPy you need, in one section

If you are new to Python, read this once. Every construct the notebook uses appears here with a small example, and each line annotation later carries its own orange **Python** box, so you can come back only when you want more.

An array is a grid of numbers with a shape

```python
import numpy as np

X = np.array([[1.0, 2.0, 3.0],
              [4.0, 5.0, 6.0]])
X.shape        # -> (2, 3)   2 rows, 3 columns
X.shape[-1]    # -> 3        the LAST axis; "-1" means "last"
X[0]           # -> [1., 2., 3.]     row 0
X[:, 0]        # -> [1., 4.]         column 0 (":" means "all of this axis")
```

Shapes are the thing to track. Almost every bug in code like this is a shape mismatch, and almost every line below can be understood by asking “what shape goes in, what shape comes out?”

The three operations that do all the work

WRITTEN AS	CALLED	WHAT IT DOES
`A @ B`	Matrix multiply	Row *i* of the result is a combination of B's rows, weighted by row *i* of A. `(n,k) @ (k,m)` → `(n,m)` — the inner numbers must match and they vanish.
`A.T`	Transpose	Flip rows and columns. `(4,8)` becomes `(8,4)`. Used to make the inner dimensions line up.
`A * B`	Elementwise	Multiply matching positions. Completely different from `@` — do not mix them up.

Why `Q @ K.T` is the shape it is: `Q` is `(seq_len, d_k)` and `K.T` is `(d_k, seq_len)`, so the result is `(seq_len, seq_len)` — one score for every ordered pair of tokens. The `d_k` disappears because it is summed over. That summing *is* the dot product.

The `axis` argument

Reduction functions collapse one axis. `axis=-1` means the last one, which for a 2-D array means *along each row*.

```
A = np.array([[1., 2., 3.],
              [4., 5., 6.]])
A.sum()               # -> 21.0        everything
A.sum(axis=-1)        # -> [6., 15.]   one number per ROW    shape (2,)
A.sum(axis=0)         # -> [5., 7., 9.] one number per COLUMN shape (3,)
A.sum(axis=-1, keepdims=True)   # -> [[6.], [15.]]  shape (2,1), NOT (2,)
```

> **KEEPDIMS IS NOT OPTIONAL DECORATION**
>
> `keepdims=True` keeps the collapsed axis as a length-1 axis. That is what lets the next line divide a `(2,3)` array by a `(2,1)` array and have NumPy stretch it across the row. Without it you get a `(2,)` array, which NumPy stretches down the *columns* instead — no error, just silently the wrong answer. This is why both `softmax` and `layer_norm` use it.

Broadcasting

When shapes differ, NumPy stretches size-1 axes to match. This is how one column of numbers gets applied to every column of a matrix:

```
A = np.array([[1., 2., 3.],
              [4., 5., 6.]])          # (2, 3)
row_max = A.max(axis=-1, keepdims=True)   # (2, 1) -> [[3.], [6.]]
A - row_max                               # (2,3) - (2,1) -> (2,3)
# [[-2., -1., 0.],
#  [-2., -1., 0.]]     each row had ITS OWN max subtracted
```

The rule: compare shapes from the right; axes must be equal, or one of them must be 1.

Adding an axis

```
pos = np.arange(4)              # [0, 1, 2, 3]        shape (4,)
pos[:, np.newaxis]              # [[0], [1], [2], [3]]  shape (4, 1)
# now  pos[:, np.newaxis] * freqs   with freqs of shape (8,)
# broadcasts to shape (4, 8): every position times every frequency
```

That single line is the whole mechanism behind positional encoding.

Strided slicing

```
v = np.array([10, 11, 12, 13, 14, 15])
v[0::2]    # -> [10, 12, 14]   start at 0, take every 2nd  (EVEN positions)
v[1::2]    # -> [11, 13, 15]   start at 1, take every 2nd  (ODD positions)
```

Used to write sines into the even columns and cosines into the odd ones.

Everything else you will meet

YOU'LL SEE	IT MEANS
`np.random.RandomState(seed)`	A private random generator. Same seed → same numbers → reproducible results.
`rng.randn(3, 4)`	A `(3,4)` array of random values, mean 0 and variance 1.
`np.zeros((5, 8))`	A `(5,8)` array of zeros. Note the *double* brackets: the shape is one tuple argument.
`np.exp`, `np.sqrt`, `np.log`	Applied to every element at once, no loop needed.
`np.maximum(0, x)`	Elementwise max against 0 — that is ReLU. Different from `np.max`, which reduces.
`np.allclose(a, b)`	Are these equal allowing for floating-point dust? Use this, never `==`, on decimals.
`np.argsort(v)`	The *indices* that would sort `v`, smallest first. `[::-1]` reverses it to largest first.
`assert cond, "msg"`	Raise an error if the condition is false. A loud, early shape check.
`x.mean(axis=-1)`	Method form of `np.mean(x, axis=-1)`. Both are common.
`a, b = x, y`	Assign two names at once.
`def f(x, n=4):`	`n` has a default, so callers may leave it out.
`f"{v:.3f}"`	An f-string: drop `v` into text with 3 decimal places.

YOU DO NOT NEED TO MEMORISE THIS

Skim it, start reading the code, and come back when a line looks strange. Every annotation from here on has its own Python box next to the line that needs it.

02 The mathematics, derived

Four questions, answered in order. By the end you will know why every symbol in the formula is there and what breaks if you remove it.

1. How do you measure “relevance”?

With a dot product. For two vectors $q, k \in \mathbb{R}^{d_k}$ the dot product is

$$q \cdot k = \sum_{i=1}^{d_k} q_i k_i = \|q\| \, \|k\| \cos\theta$$

It is large when the vectors point the same way, zero when perpendicular, negative when opposed. So it is a similarity score — and crucially it is *cheap*: one multiply-add per dimension, and a whole matrix of them is a single matrix multiply.

Stack all queries into $Q \in \mathbb{R}^{n \times d_k}$ and all keys into $K \in \mathbb{R}^{n \times d_k}$, and one operation gives every pairwise score:

$$S = QK^T \in \mathbb{R}^{n \times n}, \qquad S_{ij} = q_i \cdot k_j$$

Read S_{ij} as: *how much does token i care about token j?* The matrix is **not** symmetric in general, because caring is not mutual — a pronoun looks for its noun much harder than the noun looks for the pronoun.

2. Why divide by $\sqrt{d_k}$?

THE VARIANCE ARGUMENT

Suppose the entries of q and k are independent with mean 0 and variance 1. Each term $q_i k_i$ then has mean 0 and variance 1, and the dot product is a sum of d_k such independent terms. Variances of independent variables add:

$$\mathrm{Var}(q \cdot k) = \sum_{i=1}^{d_k} \mathrm{Var}(q_i k_i) = d_k, \qquad \mathrm{sd}(q \cdot k) = \sqrt{d_k}$$

So the raw scores grow like $\sqrt{d_k}$. At $d_k = 1024$ they typically span ±32 — and `softmax` of numbers that spread out is essentially a hard `argmax`. Dividing by $\sqrt{d_k}$ pulls the variance back to exactly 1, whatever the width:

$$\mathrm{Var}\left(\frac{q \cdot k}{\sqrt{d_k}}\right) = \frac{d_k}{d_k} = 1$$

> Section 15 measures exactly this and confirms it numerically.

3. Why softmax, and what does it actually do?

$$\text{softmax}(z)_i = \frac{e^{z_i}}{\sum_j e^{z_j}}$$

It maps any real vector to positive numbers summing to 1 — so the output is a genuine weighted *average* of the values and cannot blow up. Two properties matter here:

- **It is shift-invariant.** $\text{softmax}(z + c) = \text{softmax}(z)$ for any constant c, because e^c cancels top and bottom. This is not a curiosity: it is the licence for the `x - np.max(x)` line that prevents overflow.
- **It exaggerates.** Because it exponentiates first, a score gap of 2 becomes a weight ratio of $e^2 \approx 7.4$. Small differences in similarity become large differences in attention.

4. Why does the gradient care?

This is the part the scaling exists for. Differentiating the softmax gives

$$\frac{\partial p_i}{\partial z_j} = p_i(\delta_{ij} - p_j) \qquad \texttt{Jacobian}$$

where δ_{ij} is 1 when $i = j$ and 0 otherwise. Every entry carries a factor of p. If the softmax has saturated — one weight at 1.0, the rest at 0 — then $p(1-p) \to 0$ and **no gradient flows back to the queries and keys at all**. The layer stops learning, silently.

So $1/\sqrt{d_k}$ is not a cosmetic constant. It is what keeps the softmax in the region where it still has a derivative, no matter how wide the model gets.

> **PUTTING IT TOGETHER**
>
> Compare, scale, normalise, blend:

$$A = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right) \in \mathbb{R}^{n \times n}, \qquad \text{Attention}(Q, K, V) = AV$$

Output row i is $\sum_j A_{ij} v_j$ — token i 's new representation is a weighted blend of every token's value vector. The shapes: $(n, d_k) \cdot (d_k, n) \to (n, n) \cdot (n, d_v) \to (n, d_v)$.

03 The core code, line by line

The formula above, in eleven lines of NumPy. Nothing is hidden and nothing is imported beyond NumPy itself.

Cell 1 — softmax and scaled dot-product attention

```
import numpy as np
def softmax(x, axis=-1):
    x = x - np.max(x, axis=axis, keepdims=True)  # subtract max for numerical stabil
    e = np.exp(x)
    return e / np.sum(e, axis=axis, keepdims=True)

def scaled_dot_product_attention(Q, K, V):
    d_k = Q.shape[-1]
    scores = Q @ K.T / np.sqrt(d_k)   # similarity between every query and every key
    weights = softmax(scores, axis=-1)  # normalize into a probability distribution
    output = weights @ V              # weighted mixture of values
    return output, weights

# Quick shape/sanity check with random vectors
Q_test = np.random.randn(4, 8)
K_test = np.random.randn(4, 8)
V_test = np.random.randn(4, 8)
out, w = scaled_dot_product_attention(Q_test, K_test, V_test)
print("Output shape:", out.shape)          # (seq_len, d_v)
print("Weights shape:", w.shape)           # (seq_len, seq_len)
print("Each row of weights sums to 1:", np.allclose(w.sum(axis=1), 1.0))
```

L1

The only import. NumPy gives arrays and the `@` operator; everything else here is arithmetic.

PYTHON

`import numpy as np` loads the library under the short alias `np`, which is universal convention — you will see `np.` in front of every NumPy call.

L2

The softmax, written to work on any axis so it can be reused on 1-D vectors and 2-D score matrices alike.

PYTHON

`axis=-1` is a default value, so `softmax(x)` and `softmax(x, axis=0)` both work. `-1` means the last axis.

L3

The stability line. Subtract each row's maximum before exponentiating. Mathematically it changes nothing — softmax is shift-invariant — but numerically it is the difference between an answer and `nan`: `np.exp(1000)` overflows to infinity, and `inf/inf` is `nan`. After subtracting, the largest exponent is `exp(0) = 1`.

PYTHON

`np.max(x, axis=axis, keepdims=True)` reduces along the axis but keeps it as length 1, so `(4,8)` becomes `(4,1)`. Broadcasting then subtracts each row's own max from that row. Drop `keepdims` and NumPy would broadcast down the columns instead — wrong, and silent.

L4

Exponentiate everything. All values are now in `(0, 1]`, so no overflow is possible.

PYTHON

`np.exp` applies elementwise to the whole array at once — no loop. This is called vectorisation and it is why NumPy is fast.

L5

Divide by the row sums so each row sums to exactly 1.

PYTHON

Again `keepdims=True`, for the same broadcasting reason. `(4,8) / (4,1)` divides each row by its own total.

L7

The attention function. It takes the three matrices and returns both the output and the weights — returning the weights is what makes the heatmaps later possible.

PYTHON

A function with three required parameters. Returning two values with a comma actually returns one tuple, which the caller unpacks.

L8

Read the key/query dimension off the shape rather than passing it in, so the function cannot disagree with its own inputs.

> PYTHON
> `Q.shape` is a tuple like `(4, 8)`; `[-1]` takes the last entry. Deriving it from the data is safer than a parameter someone could set wrong.

L9

The heart of it. `Q @ K.T` computes every query-key dot product in one operation, then the scaling divides by $\sqrt{d_k}$.
Shapes: `(n, d_k) @ (d_k, n)` → `(n, n)`.

> PYTHON
> `@` is matrix multiplication (not `*`, which is elementwise). `K.T` transposes K so the inner dimensions match. Division by a plain number applies to every element.

L10

Turn scores into weights. `axis=-1` normalises *each row*, so every token's attention over all tokens sums to 1 — row-wise, not globally.

> PYTHON
> Passing `axis=-1` explicitly here even though it is the default: worth the clarity, because getting this axis wrong is the single most common bug in attention code.

L11

Blend the values. Row *i* of the output is the weighted sum of all value rows, using row *i* of the weights.
Shapes: `(n, n) @ (n, d_v)` → `(n, d_v)`.

> PYTHON
> Another matrix multiply. This is the step where information actually moves between tokens — everything before it was just deciding how much.

L12

Return both. The output feeds the next layer; the weights are for inspection.

L15–17

A shape check on random data. `4` tokens, `8` dimensions each.

> PYTHON
> `np.random.randn(4, 8)` draws from a standard normal distribution. This uses the global generator, so results vary between runs — fine here, since only shapes are being checked.

L18–21

Run it and verify three things: the output keeps the sequence length, the weights are square, and every row sums to 1.

> PYTHON

`np.allclose(a, b)` compares floating-point numbers with a tolerance. Never use `==` on decimals — `0.1 + 0.2 == 0.3` is `False` in Python.

OUTPUT

```
Output shape: (4, 8)
Weights shape: (4, 4)
Each row of weights sums to 1: True
```

THE TWO SHAPES TO REMEMBER

Weights are always `(n, n)` — one number per ordered pair of tokens, independent of the embedding width. **Output is always `(n, d_v)`** — the same shape as the input. That second fact is what makes it possible to stack these blocks, and it comes up again in Section 18.

04 Exercise — feed an array to softmax

The task: create a small array, pass it to `softmax`, and print the result.

Solution

```python
# Exercise 1 - feed a small array to softmax and look at what comes back.

a = np.array([2.0, 1.0, 0.1])
p = softmax(a)
print("input :", a)
print("softmax:", np.round(p, 4))
print("sums to:", p.sum())

# Softmax is monotonic: the largest input stays the largest output, but the
# gaps get stretched. 2.0 is 2x the size of 1.0, yet it gets ~2.7x the weight.
print("\nratio of top two weights:", round(p[0] / p[1], 3),
      " vs exp(2.0-1.0) =", round(np.exp(1.0), 3))

# axis=-1 means "normalise along the last axis", i.e. each ROW separately.
b = np.array([[1.0, 2.0, 3.0],
              [1.0, 1.0, 1.0]])
print("\n2-D input, row-wise softmax:")
print(np.round(softmax(b, axis=-1), 4))
print("row sums:", softmax(b, axis=-1).sum(axis=-1))
# Row 2 is uniform: equal scores -> equal attention. This is what an
# attention row looks like when a token has no reason to prefer anything.

# Why the function subtracts the max first (the line that looks pointless):
big = np.array([1000.0, 1001.0, 1002.0])
print("\nstable   :", np.round(softmax(big), 4))
with np.errstate(over="ignore", invalid="ignore"):
    naive = np.exp(big) / np.sum(np.exp(big))
print("naive    :", naive)
print("-> exp(1000) overflows to inf, and inf/inf is nan.")
# Subtracting the max shifts the inputs so the largest exponent is exp(0)=1.
# The result is mathematically identical because the shift cancels:
#   e^(x_i - m) / sum_j e^(x_j - m) = (e^-m . e^x_i) / (e^-m . sum_j e^x_j)
print("shifted inputs:", big - big.max(), "-> exp() of these is safe")
```

L3–7

The basic case. Three scores in, three weights out, summing to 1.

PYTHON

`np.round(p, 4)` rounds for display only — it does not change `p`. Handy when raw floats have fifteen digits of noise.

L11–13

Softmax is *monotonic but exaggerating*. The inputs 2.0 and 1.0 differ by 1, and the output weights differ by a factor of exactly $e \approx 2.718$. That is not a coincidence — it is the definition: the ratio of two weights is $e^{z_i - z_j}$.

PYTHON

The `\n` at the start of a string prints a blank line first. `round(x, 3)` is Python's own function, distinct from `np.round` which works on whole arrays.

L16–21

The 2-D case, which is what attention actually uses. `axis=-1` normalises each row independently, so each row becomes its own distribution. Note the second row: **equal scores give exactly uniform weights**. That is what an attention row looks like when a token has no reason to prefer anything.

PYTHON

A 2-D array is written as a list of lists. Passing `axis=-1` collapses along the last axis; the row sums confirm it worked row-wise rather than over the whole array.

L25–29

Why line 3 of the original function exists. With inputs near 1000 the naive formula returns `nan`: `np.exp(1000)` overflows to `inf`, and `inf/inf` is undefined. The library version returns the right answer.

PYTHON

`np.errstate(over="ignore")` temporarily silences NumPy's overflow warning so the demonstration prints cleanly — the overflow still happens, it just is not reported.

L30–35

The justification: subtracting a constant leaves softmax unchanged, because the e^{-m} factor cancels top and bottom. So we are free to subtract the max, which forces the largest exponent to be $e^{0} = 1$.

OUTPUT

```
input : [2.  1.  0.1]
softmax: [0.659  0.2424 0.0986]
sums to: 1.0

ratio of top two weights: 2.718  vs exp(2.0-1.0) = 2.718

2-D input, row-wise softmax:
[[0.09   0.2447 0.6652]
 [0.3333 0.3333 0.3333]]
row sums: [1. 1.]

stable   : [0.09   0.2447 0.6652]
naive    : [nan nan nan]
-> exp(1000) overflows to inf, and inf/inf is nan.
shifted inputs: [-2. -1.  0.] -> exp() of these is safe
```

THE SHIFT-INVARIANCE IDENTITY

The line that looks pointless is doing all the safety work. Formally:

$$\frac{e^{z_i - m}}{\sum_j e^{z_j - m}} = \frac{e^{-m} e^{z_i}}{e^{-m} \sum_j e^{z_j}} = \frac{e^{z_i}}{\sum_j e^{z_j}}$$

Mathematically identical, numerically the difference between an answer and `nan`. Choosing $m = \max_j z_j$ makes the largest exponent $e^0 = 1$, so nothing can overflow.

05 Exercise — build Q, K, V by hand

The task: make three arrays, pass them to `scaled_dot_product_attention`, and observe the result. The vectors below are chosen so you can predict every row before running it.

Solution

```python
# Exercise 2 - build Q, K, V by hand and watch attention move information around.
#
# 3 "tokens", 4 dimensions. The vectors are deliberately readable rather than
# random, so you can predict the answer before running it.

Q_ex = np.array([[1.0, 0.0, 0.0, 0.0],     # token 0 asks for "feature 0"
                 [0.0, 1.0, 0.0, 0.0],     # token 1 asks for "feature 1"
                 [0.0, 0.0, 1.0, 0.0]])    # token 2 asks for "feature 2"

K_ex = np.array([[1.0, 0.0, 0.0, 0.0],     # token 0 advertises feature 0
                 [0.0, 1.0, 0.0, 0.0],     # token 1 advertises feature 1
                 [1.0, 0.0, 0.0, 0.0]])    # token 2 ALSO advertises feature 0

V_ex = np.array([[10.0, 0.0, 0.0, 0.0],    # the payload each token hands over
                 [0.0, 20.0, 0.0, 0.0],
                 [0.0, 0.0, 30.0, 0.0]])

out_ex, w_ex = scaled_dot_product_attention(Q_ex, K_ex, V_ex)

print("raw scores Q @ K.T:")
print(Q_ex @ K_ex.T)
print("\nafter dividing by sqrt(d_k) =", np.sqrt(4))
print(np.round(Q_ex @ K_ex.T / np.sqrt(4), 3))
print("\nattention weights (each row sums to 1):")
print(np.round(w_ex, 4))
print("\noutput:")
print(np.round(out_ex, 3))

# Reading it:
# Row 0 - query 0 matches keys 0 AND 2 (both advertise feature 0), so its
#         attention splits between them and its output mixes V[0] and V[2].
# Row 1 - query 1 matches only key 1, so it is the peakiest row.
# Row 2 - query 2 matches nothing, so all three scores are 0 and the softmax
#         of equal scores is exactly uniform: it averages all three values.
print("\nrow 2 weights are uniform:", np.allclose(w_ex[2], 1/3))
print("row 2 output equals the plain mean of V:", np.allclose(out_ex[2], V_ex.mean(a

# The output really is just a weighted sum of the value ROWS - nothing else.
manual = w_ex[0, 0] * V_ex[0] + w_ex[0, 1] * V_ex[1] + w_ex[0, 2] * V_ex[2]
print("\nrow 0 recomputed by hand:", np.round(manual, 3))
print("matches the function's output:", np.allclose(manual, out_ex[0]))
```

L6–16

Three tokens, four dimensions. Each query is a one-hot vector asking for one feature; each key advertises one feature; each value is the payload that token contributes. Note key 2 deliberately advertises the *same* feature as key 0.

PYTHON

`np.array` of a list of lists gives a `(3,4)` array. Writing the rows on separate lines is purely for readability — Python ignores line breaks inside brackets.

L18

One call, two returns.

PYTHON

Tuple unpacking: the function returns `(output, weights)` and this splits them into two names.

L20–29

Print each stage: raw dot products, then scaled, then weights, then output. Seeing the intermediate matrices is the fastest way to build intuition.

PYTHON

`Q_ex @ K_ex.T` re-does by hand what the function does internally — useful for checking your mental model against the code.

L31–41

The three readings, and two verifications. Row 2's query matches nothing, so all its scores are 0 and softmax of equal scores is exactly uniform — its output is the plain mean of V. Row 0 matches two keys and splits between them.

PYTHON

`V_ex.mean(axis=0)` averages down the *columns* (across rows), giving the mean value vector. `np.allclose` confirms the match despite floating-point dust.

OUTPUT

```
raw scores Q @ K.T:
[[1. 0. 1.]
 [0. 1. 0.]
 [0. 0. 0.]]

after dividing by sqrt(d_k) = 2.0
[[0.5 0.  0.5]
 [0.  0.5 0. ]
 [0.  0.  0. ]]

attention weights (each row sums to 1):
[[0.3837 0.2327 0.3837]
 [0.2741 0.4519 0.2741]
 [0.3333 0.3333 0.3333]]

output:
[[ 3.837  4.654 11.51   0.   ]
 [ 2.741  9.037  8.222  0.   ]
 [ 3.333  6.667 10.     0.   ]]

row 2 weights are uniform: True
row 2 output equals the plain mean of V: True

row 0 recomputed by hand: [ 3.837  4.654 11.51   0.   ]
matches the function's output: True
```

Reading the output

ROW	WHAT ITS QUERY ASKS	RESULT
0	feature 0	Keys 0 *and* 2 both advertise it, so attention splits 0.38 / 0.23 / 0.38 and the output mixes `V[0]` and `V[2]`.
1	feature 1	Only key 1 matches, so this is the peakiest row at 0.45.
2	feature 2	Nothing matches. All scores are 0, weights are exactly ⅓ each, and the output is the plain average of all values.

THE ONE THING TO TAKE AWAY

The final line proves it explicitly: the output row is *nothing more* than $\sum_j A_{ij} v_j$ — a weighted sum of the value rows. All the machinery upstream exists only to decide

the weights A_{ij} . Attention does not create information; it **routes** it.

06 The worked example: making “it” find its noun

The sentence is *“The animal didn't cross the street because it was tired.”* We want to watch attention link **“it” → “animal”** rather than “it” → “street”.

READ THE HONESTY NOTE IN THE NOTEBOOK

With *random, untrained* weights this link would not reliably appear — the model has learned nothing yet. So the notebook hand-crafts small feature vectors instead. This demonstrates what the **mechanism** does given meaningful input; it is not evidence that an untrained transformer resolves coreference. A trained model learns embeddings that play the role these hand-made features play here.

Cell 2 — the worked example

```python
tokens_list = ["The", "animal", "didn't", "cross", "the", "street", "because", "it",

# Hand-crafted feature embeddings -- NOT random, NOT learned.
# dims: [is_entity, is_animate, is_place, is_function_word, is_verb, is_adjective, p
feats = {
    "The":     [0, 0, 0, 1, 0, 0, 0, 0],
    "animal":  [1, 1, 0, 0, 0, 0, 0, 0],
    "didn't":  [0, 0, 0, 1, 0, 0, 0, 0],
    "cross":   [0, 0, 0, 0, 1, 0, 0, 0],
    "the":     [0, 0, 0, 1, 0, 0, 0, 0],
    "street":  [1, 0, 1, 0, 0, 0, 0, 0],
    "because": [0, 0, 0, 1, 0, 0, 0, 0],
    "it":      [1, 1, 0, 0, 0, 0, 0, 0],   # shares [entity, animate] with "animal"
    "was":     [0, 0, 0, 1, 0, 0, 0, 0],
    "tired":   [0, 0, 0, 0, 0, 1, 0, 0],
}
X = np.array([feats[t] for t in tokens_list], dtype=float)

# Using identity-like projections here (Q = K = V = X) keeps the demo transparent --
# in a real model, W_q/W_k/W_v are learned matrices that would produce similar
# alignments automatically once trained on enough language data.
Q, K, V = X, X, X
output, attn_weights = scaled_dot_product_attention(Q, K, V)

it_idx = tokens_list.index("it")
order = np.argsort(attn_weights[it_idx])[::-1]
print("Token 'it' attends to (ranked by weight):")
for i in order[:4]:
    print(f"  {tokens_list[i]:10s}  weight={attn_weights[it_idx][i]:.3f}")
```

L1

The sentence as a list of tokens.

PYTHON

A plain Python list of strings. Real models split text into sub-word pieces, but the mechanism is identical.

L3–4

The comment declares what each of the eight dimensions means. This is the interpretive key for everything below.

L5–16

A dict from word to feature vector. The load-bearing line is `"it"`: it is given `[1,1,0,...]`, the same `[entity, animate]` pattern as `"animal"`. Two zero “pad” dimensions bring the width to 8 so it divides evenly by the head count later.

PYTHON

A dict maps keys to values. Here each value is a list of 8 numbers, written to line up visually so the columns are readable.

L17

Build the embedding matrix by looking up each token in order. Shape `(10, 8)`: ten tokens, eight features.

PYTHON

A list comprehension: `[feats[t] for t in tokens_list]` builds a list of lists, which `np.array` turns into a 2-D array. `dtype=float` forces decimals — without it you would get integers and later divisions would behave differently.

L19–22

`Q = K = V = X`. In a real model these are `X @ W_q`, `X @ W_k`, `X @ W_v` with learned matrices. Setting them all to X removes that layer of indirection so you can see the mechanism unobscured.

PYTHON

One line assigning three names to the same array. They are three references to one object, not three copies — fine here because nothing modifies them in place.

L23

Run attention over the sentence.

L25–29

Find the row for `"it"` and rank what it attends to.

PYTHON

`.index("it")` finds a value's position in a list. `np.argsort` returns the indices that would sort the array ascending; `[::-1]` reverses that to descending; `[:4]` takes the top four.

OUTPUT

```
Token 'it' attends to (ranked by weight):
  animal      weight=0.163
  it          weight=0.163
  street      weight=0.114
  tired       weight=0.080
```

Why it works, arithmetically

Because $Q = K = X$, the score between two tokens is just the dot product of their feature vectors — which counts **how many features they share**:

PAIR	SHARED FEATURES	RAW SCORE
`it` · `animal`	entity, animate	**2**
`it` · `it`	entity, animate	**2**
`it` · `street`	entity only	1
`it` · `tired`	none	0

Scores of 2, 2, 1, 0 become weights 0.163, 0.163, 0.114, 0.080 — exactly the ranking in the output. “animal” ties with “it” itself and clearly beats “street”.

> **WHY THE WEIGHTS LOOK SO FLAT**
>
> The top weight is only 0.163, not 0.9. Two reasons, both worth understanding: the scores are tiny integers (0, 1, 2) so e^{Δ} is at most about 7, and there are ten tokens sharing the probability mass — uniform would be 0.100. A trained model produces far larger score gaps and correspondingly peakier attention. What matters here is the *ranking*, not the magnitude.

07 Exercise — your own sentence

The task: make your own token list, create an encoding for every token, and compute the attention.

I chose *“The dog chased the ball because it was rolling”* — deliberately the mirror image of the original. Here “it” refers to the **inanimate** thing. If the mechanism is

real rather than rigged, attention should now prefer “ball” over “dog”.

Solution

```python
# Exercise - your own sentence, your own features, your own attention.
#
# Sentence: "The dog chased the ball because it was rolling."
# This is the mirror image of the lecture's example. Here "it" refers to the
# INANIMATE thing, so if the mechanism is real (and not us having rigged the
# original), attention should now prefer "ball" over "dog".

tokens_list_demo = ["The", "dog", "chased", "the", "ball",
                    "because", "it", "was", "rolling"]

# dims: [is_entity, is_animate, is_object, is_function_word, is_verb, is_adjective,
feats_demo = {
    "The":     [0, 0, 0, 1, 0, 0, 0, 0],
    "dog":     [1, 1, 0, 0, 0, 0, 0, 0],   # entity + animate
    "chased":  [0, 0, 0, 0, 1, 0, 0, 0],
    "the":     [0, 0, 0, 1, 0, 0, 0, 0],
    "ball":    [1, 0, 1, 0, 0, 0, 0, 0],   # entity + object
    "because": [0, 0, 0, 1, 0, 0, 0, 0],
    "it":      [1, 0, 1, 0, 0, 0, 0, 0],   # entity + object  -> matches "ball"
    "was":     [0, 0, 0, 1, 0, 0, 0, 0],
    "rolling": [0, 0, 0, 0, 1, 0, 0, 0],
}
X_demo = np.array([feats_demo[t] for t in tokens_list_demo], dtype=float)

# NOTE: the skeleton in the notebook says `Q, K, V = X, X, X`, which quietly
# reuses the PREVIOUS sentence's matrix and computes the wrong thing entirely.
# It has to be X_demo.
Q_demo, K_demo, V_demo = X_demo, X_demo, X_demo
output_demo, attn_weights_demo = scaled_dot_product_attention(Q_demo, K_demo, V_demo

it_demo = tokens_list_demo.index("it")
order = np.argsort(attn_weights_demo[it_demo])[::-1]
print("Token 'it' attends to (ranked):")
for i in order[:5]:
    print(f"  {tokens_list_demo[i]:10s} weight={attn_weights_demo[it_demo][i]:.3f}")

w_ball = attn_weights_demo[it_demo][tokens_list_demo.index("ball")]
w_dog = attn_weights_demo[it_demo][tokens_list_demo.index("dog")]
print(f"\n'ball' {w_ball:.3f}  vs  'dog' {w_dog:.3f}   ->  ratio {w_ball/w_dog:.2f}x
print("The raw scores behind that: it.ball =", X_demo[it_demo] @ feats_demo["ball"],
      " it.dog =", X_demo[it_demo] @ np.array(feats_demo["dog"], dtype=float))
print("\nSame mechanism, opposite answer - because the FEATURES changed, not the cod
```

L1–9	The setup, and the hypothesis being tested.
L11–12	The new sentence. Nine tokens this time — nothing depends on the length.
L14–26	New feature scheme: dimension 2 is now `is_object` rather than `is_place`. `"ball"` and `"it"` both get `[entity, object]`; `"dog"` gets `[entity, animate]`. So `it·ball = 2` but `it·dog = 1`. PYTHON Same dict-of-lists structure as before. The keys must exactly match the strings in the token list, or the lookup on the next line raises `KeyError`.
L28–33	**A bug in the notebook's skeleton.** The provided cell says `Q, K, V = X, X, X` — the *old* sentence's matrix. It runs without error and produces a `(10,10)` attention matrix for a 9-token sentence, which then fails or silently misleads. It must be `X_demo`. PYTHON This is the classic copy-paste hazard: a name that still exists from an earlier cell, so Python raises nothing. Notebooks make this especially easy because state persists across cells.
L35–44	Rank what “it” attends to, then print the ratio and the underlying raw scores so the result is traceable to arithmetic rather than taken on faith. PYTHON An f-string with a format spec: `{w_ball:.3f}` prints three decimal places. The `@` between two 1-D arrays computes their dot product and returns a single number.

OUTPUT

```
Token 'it' attends to (ranked):
  it          weight=0.177
  ball        weight=0.177
  dog         weight=0.124
  rolling     weight=0.087
  was         weight=0.087

'ball' 0.177  vs  'dog' 0.124   ->  ratio 1.42x
The raw scores behind that: it.ball = 2.0  it.dog = 1.0

Same mechanism, opposite answer - because the FEATURES changed, not the code.
```

“ball” 0.177 versus “dog” 0.124 — a 1.42× preference, from raw scores of 2 versus 1. Same code, opposite antecedent, because the features changed and nothing else did.

WHAT THIS ACTUALLY DEMONSTRATES

That attention is a *mechanism*, not a knowledge base. It faithfully computes similarity in whatever feature space you hand it. Get the features right and coreference falls out; get them wrong and it will confidently attend to the wrong word. In a real transformer, learning good features *is* the training problem — the attention arithmetic never changes.

08 Visualising the attention matrix

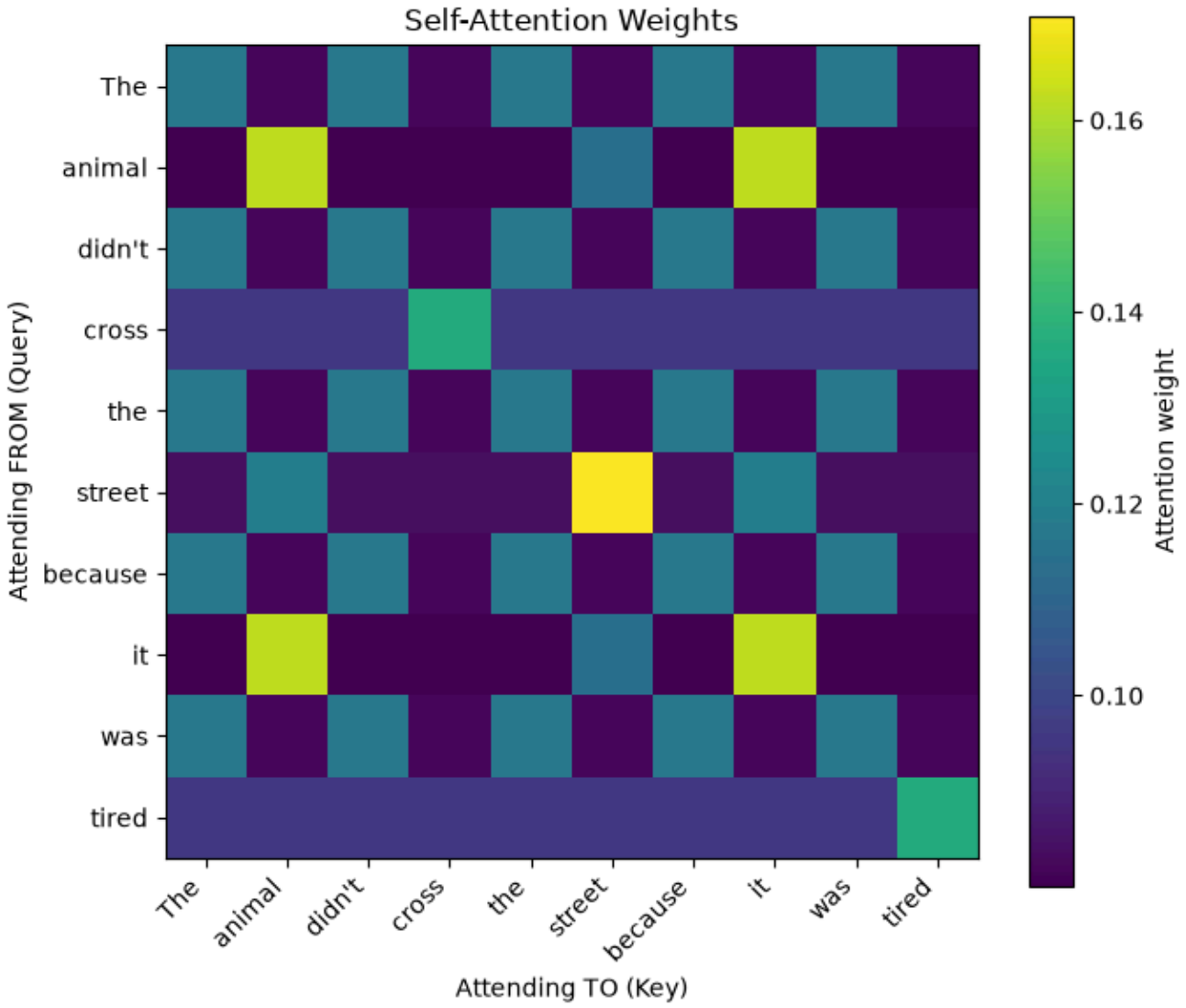
A 10×10 table of decimals is hard to scan. The same numbers as a heatmap are readable instantly.

Cell 3 — the heatmap

```
import matplotlib.pyplot as plt

fig, ax = plt.subplots(figsize=(7, 6))
im = ax.imshow(attn_weights, cmap="viridis")
ax.set_xticks(range(len(tokens_list)))
ax.set_yticks(range(len(tokens_list)))
ax.set_xticklabels(tokens_list, rotation=45, ha="right")
ax.set_yticklabels(tokens_list)
ax.set_xlabel("Attending TO (Key)")
ax.set_ylabel("Attending FROM (Query)")
ax.set_title("Self-Attention Weights")
fig.colorbar(im, ax=ax, label="Attention weight")
plt.tight_layout()
plt.savefig("attention_heatmap.png", dpi=120)
plt.show()
```

L1	matplotlib's plotting interface. PYTHON `import matplotlib.pyplot as plt` — `plt` is the universal alias.
L3	Create a figure and one set of axes. `fig` is the whole canvas, `ax` is the plot area. PYTHON `plt.subplots()` returns a `(figure, axes)` tuple, unpacked into two names. `figsize` is in inches.
L4	Draw the matrix as an image: one pixel per cell, colour-mapped by value. PYTHON `imshow` means “show image”. `cmap="viridis"` selects a perceptually uniform colour map — dark blue for low, yellow for high — which is readable in greyscale and to colour-blind viewers.
L5–8	Put the actual words on both axes instead of index numbers. PYTHON `set_xticks` chooses *where* the labels go, `set_xticklabels` chooses *what* they say — you need both. `rotation=45, ha="right"` angles the x labels so they do not collide.
L9–11	Label the axes with the direction of attention. This matters: the matrix is not symmetric, so which axis is the query and which is the key is essential information.
L12	A colour bar, so the reader can map colour back to number.
L13–15	Tidy the spacing, save at higher resolution, display. PYTHON `tight_layout()` stops the rotated labels being clipped. `dpi=120` makes the saved PNG sharper than the default 100.



The original sentence. Every row sums to 1. The bright block in the middle is the four function words — `The`, `the`, `because`, `was` — which share an identical feature vector and so cannot be told apart.

The exercise: your own sentence

The task: visualise the demo attention matrix.

Solution

```python
# Exercise - visualise your own sentence's attention matrix.

fig, ax = plt.subplots(figsize=(7, 6))
im = ax.imshow(attn_weights_demo, cmap="viridis")
ax.set_xticks(range(len(tokens_list_demo)))
ax.set_yticks(range(len(tokens_list_demo)))
ax.set_xticklabels(tokens_list_demo, rotation=45, ha="right")
ax.set_yticklabels(tokens_list_demo)
ax.set_xlabel("Attending TO (Key)")
ax.set_ylabel("Attending FROM (Query)")
ax.set_title("Self-Attention: 'The dog chased the ball because it was rolling'")

# Mark the row we care about so the coreference link is impossible to miss.
ax.add_patch(plt.Rectangle((-0.5, it_demo - 0.5), len(tokens_list_demo), 1,
                           fill=False, edgecolor="red", lw=2))
fig.colorbar(im, ax=ax, label="Attention weight")
plt.tight_layout()
plt.savefig("attention_heatmap_demo.png", dpi=120)
plt.show()

# The visible structure is block-like: all four function words ("The", "the",
# "because", "was") have identical feature vectors, so they form one bright
# block that attends to itself. Attention cannot tell them apart at all --
# which is exactly the job positional encoding exists to fix.
identical = np.allclose(attn_weights_demo[0], attn_weights_demo[3])
print("rows for 'The' and 'the' are identical:", identical)
```

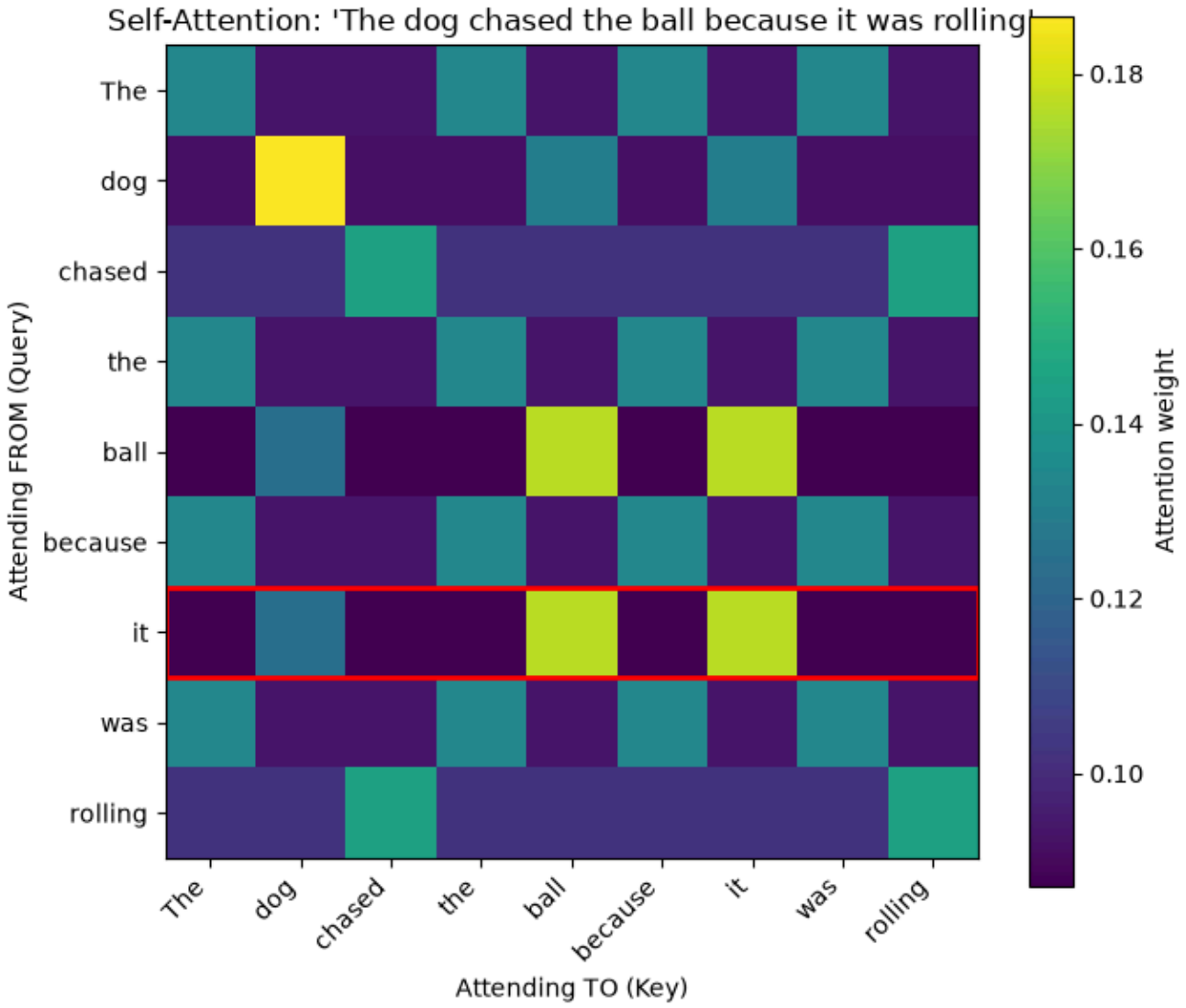
L3–13 The same plotting recipe, pointed at `attn_weights_demo` and the new token list.

L15–18 An addition worth making: draw a red rectangle around the `"it"` row so the coreference link is impossible to miss.

> PYTHON
> `plt.Rectangle((x, y), width, height)` takes the bottom-left corner first. The `-0.5` offsets exist because `imshow` centres each cell on its integer index, so the cell edges fall on the halves.

L19–23 Colour bar, layout, save, show.

L25–29 A check that makes the visual claim precise: the rows for `"The"` and `"the"` are *identical*, confirming attention alone cannot distinguish two occurrences of the same word.



The mirror-image sentence, with the `it` row outlined. Its brightest cells are `ball` and `it` — not `dog`.

WHAT THE PICTURE PROVES

Identical rows for `The` and `the` are not a rendering artefact — `np.allclose` confirms it. Self-attention is **permutation-equivariant**: shuffle the input tokens and the output rows shuffle identically, but nothing else changes. The mechanism has no concept of first, second or last.

That is a fatal flaw for language, and it is precisely the hole that positional encoding — Section 11 — exists to fill.

09 Multi-head attention

One attention computation can measure exactly one kind of similarity.

Language needs several at once — who did what, which noun a pronoun refers

to, what the topic is. The fix is embarrassingly simple: run several attentions in parallel and concatenate.

THE FORMULA

Each head gets its own learned projections, so each looks at a different subspace:

$$\text{head}_h = \text{Attention}(XW_h^Q,\ XW_h^K,\ XW_h^V)$$

and the heads are concatenated and projected back to the model width:

$$\text{MultiHead}(X) = \text{Concat}(\text{head}_1,\ \dots,\ \text{head}_H)\, W^O$$

with $d_{\text{head}} = d_{\text{model}}/H$, so that $H \cdot d_{\text{head}} = d_{\text{model}}$ and the output is (n, d_{model}) — the same shape it went in as.

Cell 4 — multi-head attention

```python
def multi_head_attention(X, num_heads, d_model, seed=0):
    rng = np.random.RandomState(seed)
    assert d_model % num_heads == 0, "d_model must be divisible by num_heads"
    d_head = d_model // num_heads

    head_outputs = []
    all_weights = []
    for h in range(num_heads):
        W_q = rng.randn(X.shape[1], d_head) * 0.3
        W_k = rng.randn(X.shape[1], d_head) * 0.3
        W_v = rng.randn(X.shape[1], d_head) * 0.3
        Qh, Kh, Vh = X @ W_q, X @ W_k, X @ W_v
        out_h, w_h = scaled_dot_product_attention(Qh, Kh, Vh)
        head_outputs.append(out_h)
        all_weights.append(w_h)

    concatenated = np.concatenate(head_outputs, axis=-1)   # (seq_len, num_heads * d
    W_o = rng.randn(concatenated.shape[1], d_model) * 0.3   # final linear projectio
    output = concatenated @ W_o
    return output, all_weights

mh_output, head_weights = multi_head_attention(X, num_heads=4, d_model=8, seed=1)
print("Multi-head output shape:", mh_output.shape)
print("Number of heads:", len(head_weights))
print("Each head's attention matrix shape:", head_weights[0].shape)

# Do different heads attend differently? Compare where 'it' looks for head 0 vs head
for h in range(2):
    order = np.argsort(head_weights[h][it_idx])[::-1][:3]
    print(f"Head {h}: 'it' -> {[tokens_list[i] for i in order]}")
```

L1–2

A private random generator seeded per call, so the “learned” projections are reproducible.

PYTHON

`np.random.RandomState(seed)` makes an isolated generator. Using it instead of `np.random.randn` means this function's randomness cannot be disturbed by anything else in the notebook.

L3

The divisibility check, as an assert so it fails loudly and early rather than producing a silently wrong shape.

PYTHON

`assert condition, "message"` raises `AssertionError` with that message when the condition is false. `%` is the remainder operator: `8 % 4` is 0.

L4

Split the model width across the heads.

PYTHON

`//` is integer division: it discards any remainder rather than producing a decimal. The assert on the previous line is what guarantees there is no remainder to discard.

L6–7

Two empty lists to collect per-head results.

L8

Loop once per head.

PYTHON

`for h in range(num_heads)` counts 0, 1, 2, ... The loop body is what runs in parallel on real hardware; here it is sequential, which changes the speed but not the result.

L9–11

The three projection matrices for this head, shaped `(d_model, d_head)` so they map the input down into this head's subspace. In a real transformer these are the *learned parameters* — here they are random, which is why the heads below specialise arbitrarily rather than meaningfully.

PYTHON

`rng.randn(a, b)` gives an `(a,b)` array with mean 0 and variance 1. Multiplying by `0.3` shrinks the scale, a crude stand-in for the careful initialisation real models use.

L12

Project the input into this head's smaller space. `(n, d_model) @ (d_model, d_head)` → `(n, d_head)`.

PYTHON

Three matrix multiplies assigned to three names on one line.

L13–15 Run the *same* attention function from Section 3 — multi-head attention adds no new mathematics, it just calls the existing operation several times on smaller inputs.

L17 Glue the heads side by side along the feature axis: `H` blocks of `(n, d_head)` become `(n, H×d_head)` = `(n, d_model)`.

> PYTHON
> `np.concatenate(list_of_arrays, axis=-1)` joins along the last axis. Use `axis=0` and you would stack them vertically instead — a common and confusing bug.

L18–19 The output projection W^O. It lets the model *mix* the heads rather than leaving them as independent blocks; without it each head's output would sit in its own slice of the vector, never interacting.

> PYTHON
> Shape `(d_model, d_model)` here, so the output width matches the input width.

L20 Return the output and every head's weights, so you can compare what different heads looked at.

L22–26 Run it with 4 heads on the 8-dimensional example and print the shapes.

L28–31 The interesting check: do different heads attend differently? Compare where `"it"` looks in head 0 versus head 1.

> PYTHON
> A list comprehension inside an f-string: `{[tokens_list[i] for i in order]}` builds and prints the list of words in one expression.

OUTPUT

```
Multi-head output shape: (10, 8)
Number of heads: 4
Each head's attention matrix shape: (10, 10)
Head 0: 'it' -> ['cross', 'tired', 'animal']
Head 1: 'it' -> ['animal', 'it', 'street']
```

Head 0 sends “it” to `['cross', 'tired', 'animal']`; head 1 sends it to `['animal', 'it', 'street']`. Genuinely different views of the same sentence.

> DO NOT OVER-READ THIS
>
> These projections are **random and untrained**, so the heads differ for the same reason any two random projections differ — not because one has discovered syntax

and another coreference. In a trained model heads do reliably specialise, and that is a real and well-documented finding; but this cell demonstrates the *capacity* for different heads to see differently, not that specialisation has occurred here.

WHY NOT JUST USE ONE BIG HEAD?

Because a single d_{model} attention has one score matrix, and one score matrix can encode only one similarity structure. Splitting into H heads gives H independent score matrices for essentially the same arithmetic cost — $H \times n^2 \times d_{\text{head}} = n^2 d_{\text{model}}$ multiplications either way.

The trade is resolution for breadth: each head measures similarity in a d_{model}/H-dimensional space instead of d_{model}. Section 17 shows what happens when you push that too far.

10 Exercise — change the number of heads

The task: change `num_heads=4` to `2`. What happens to `d_head`? Then try 4 again and observe.

Solution

```python
# Exercise - change num_heads and watch d_head move.
#
#   d_head = d_model // num_heads
#
# d_model is the width of the model and is FIXED. Splitting it into more heads
# does not add capacity, it re-slices the same capacity into thinner pieces.

print(f"{'num_heads':>10s}{'d_head':>8s}{'concat width':>14s}{'output shape':>15s}")
print("-" * 47)
for nh in (1, 2, 4, 8):
    out_nh, w_nh = multi_head_attention(X, num_heads=nh, d_model=8, seed=1)
    d_head = 8 // nh
    print(f"{nh:>10d}{d_head:>8d}{nh * d_head:>14d}{str(out_nh.shape):>15s}")

print("""
num_heads=4 -> d_head=2      num_heads=2 -> d_head=4

Two things stay constant no matter how you slice it:
  * num_heads x d_head is always d_model (8), because the heads are
    CONCATENATED - that is what keeps the output shape at (10, 8) and lets
    blocks stack.
  * the total amount of attention arithmetic is roughly unchanged.

What changes is the trade-off: more heads = more independent relationships
the layer can look for at once, but each one gets a lower-resolution space to
measure similarity in.""")

# Do fewer, fatter heads actually behave differently? Compare where 'it' looks.
for nh in (2, 4):
    _, w_nh = multi_head_attention(X, num_heads=nh, d_model=8, seed=1)
    print(f"\nnum_heads={nh}:")
    for h in range(nh):
        top = np.argsort(w_nh[h][it_idx])[::-1][:3]
        print(f"  head {h}: 'it' -> {[tokens_list[i] for i in top]}")
```

L1–7	The relationship, stated up front: `d_head = d_model // num_heads`. `d_model` is fixed, so more heads does not mean more capacity — it means the same capacity sliced thinner.
L9–16	Rather than editing the number by hand and re-running, sweep all four values and tabulate. Faster, and it makes the invariant visible. PYTHON `for nh in (1, 2, 4, 8)` loops over a tuple of values. Formatting each row with `{nh:>10d}` right-aligns in 10 columns so the table lines up.
L18–29	The two invariants and the trade-off they imply.
L31–37	Do fewer, fatter heads behave differently? Print where `"it"` looks under each configuration. PYTHON `_, w_nh = ...` discards the first return value with `_`, the conventional name for “I must name this but will not use it”.

OUTPUT

```
 num_heads  d_head  concat width   output shape
-----------------------------------------------
         1       8             8        (10, 8)
         2       4             8        (10, 8)
         4       2             8        (10, 8)
         8       1             8        (10, 8)

num_heads=4 -> d_head=2      num_heads=2 -> d_head=4

Two things stay constant no matter how you slice it:
  * num_heads x d_head is always d_model (8), because the heads are
    CONCATENATED - that is what keeps the output shape at (10, 8) and lets
    blocks stack.
  * the total amount of attention arithmetic is roughly unchanged.

What changes is the trade-off: more heads = more independent relationships
the layer can look for at once, but each one gets a lower-resolution space to
measure similarity in.

num_heads=2:
  head 0: 'it' -> ['cross', 'was', 'The']
  head 1: 'it' -> ['animal', 'it', 'The']

num_heads=4:
  head 0: 'it' -> ['cross', 'tired', 'animal']
  head 1: 'it' -> ['animal', 'it', 'street']
  head 2: 'it' -> ['tired', 'cross', 'was']
  head 3: 'it' -> ['was', 'because', 'The']
```

The answer

NUM_HEADS	D_HEAD	CONCAT WIDTH	OUTPUT SHAPE
1	8	8	`(10, 8)`
2	**4**	8	`(10, 8)`
4	**2**	8	`(10, 8)`
8	1	8	`(10, 8)`

4 heads → `d_head=2`**; 2 heads →** `d_head=4`**.** The product is always 8, and the output shape never moves — which is exactly what lets an encoder block be

stacked.

THE REAL TRADE-OFF

More heads means more independent relationships the layer can look for simultaneously, but each one measures similarity in a lower-dimensional space. Real models keep `d_head` around 64–128 and scale the head count with width: GPT-3 uses 96 heads of 128 dimensions (`d_model = 12288`). Here, with `d_model=8` , every option is far below that — which is why Section 17's `d_head=1` degenerates so visibly.

11 Positional encoding

Section 8 proved that attention cannot tell two identical tokens apart. Everything here exists to fix that.

WHY ORDER IS INVISIBLE TO ATTENTION

Attention computes $q_i \cdot k_j$ and blends v_j . Permute the tokens with a permutation matrix P and every term moves with them:

$$\text{Attention}(PQ, PK, PV) = P \cdot \text{Attention}(Q, K, V)$$

The output permutes but never *changes*. “Dog bites man” and “man bites dog” would produce the same set of representations. Since word order carries meaning, position must be injected into the input itself — and it is, by plain addition: $x = \text{embedding} + PE$.

The formula

$$PE(pos, 2i) = \sin\left(\frac{pos}{10000^{2i/d}}\right), \qquad PE(pos, 2i+1) = \cos\left(\frac{pos}{10000^{2i/d}}\right)$$

Read it as a set of clocks. Dimension pair i ticks at angular frequency $\omega_i = 10000^{-2i/d}$. Pair 0 runs fastest (period $2\pi \approx 6.28$ positions); the last pair runs slowest (period tens of thousands). Every position gets a unique combination of hand positions — like reading a row of clocks with wildly different speeds.

Cell 5 — sinusoidal positional encoding

```
def positional_encoding(seq_len, d_model):
    position = np.arange(seq_len)[:, np.newaxis]
    div_term = np.exp(np.arange(0, d_model, 2) * -(np.log(10000.0) / d_model))
    pe = np.zeros((seq_len, d_model))
    pe[:, 0::2] = np.sin(position * div_term)
    pe[:, 1::2] = np.cos(position * div_term)
    return pe

pe = positional_encoding(seq_len=50, d_model=64)
print("Positional encoding shape:", pe.shape)

fig, ax = plt.subplots(figsize=(8, 5))
im = ax.imshow(pe.T, cmap="RdBu", aspect="auto")
ax.set_xlabel("Position in sequence")
ax.set_ylabel("Embedding dimension")
ax.set_title("Sinusoidal Positional Encoding")
fig.colorbar(im, ax=ax, label="Value")
plt.tight_layout()
plt.savefig("positional_encoding.png", dpi=120)
plt.show()
```

L1

Takes how many positions you need and how wide the vectors must be.

L2

A column of position indices, shape `(seq_len, 1)`.

PYTHON

`np.arange(seq_len)` gives `[0,1,2,...]` with shape `(seq_len,)`. `[:, np.newaxis]` adds a second axis, turning the row into a column — which is what makes the broadcast on lines 5–6 produce a 2-D grid.

L3

The frequencies, one per dimension pair. Written as $e^{-2i\ln(10000)/d}$ using the identity $a^b = e^{b\ln a}$, which is numerically steadier than raising 10000 to a negative power directly.

PYTHON

`np.arange(0, d_model, 2)` counts `0, 2, 4, ...` — the third argument is the step. `np.log` is the natural logarithm (base *e*), not base 10.

L4

An empty array of the right shape, to be filled in.

PYTHON

`np.zeros((seq_len, d_model))` — note the double brackets, because the shape is a single tuple argument.

L5

Write sines into the **even** columns. `(seq_len,1) * (d_model/2,)` broadcasts to `(seq_len, d_model/2)`: every position times every frequency.

PYTHON

`pe[:, 0::2]` selects all rows and every second column starting at 0. Assigning to a slice writes into the existing array in place.

L6

Cosines into the **odd** columns, using the same frequencies. Each consecutive pair of columns is therefore a (sin, cos) pair sharing one clock — which is what makes the rotation property below work.

L7

Return the `(seq_len, d_model)` table.

L9–10

Build a 50-position, 64-dimensional encoding and confirm the shape.

L12–20

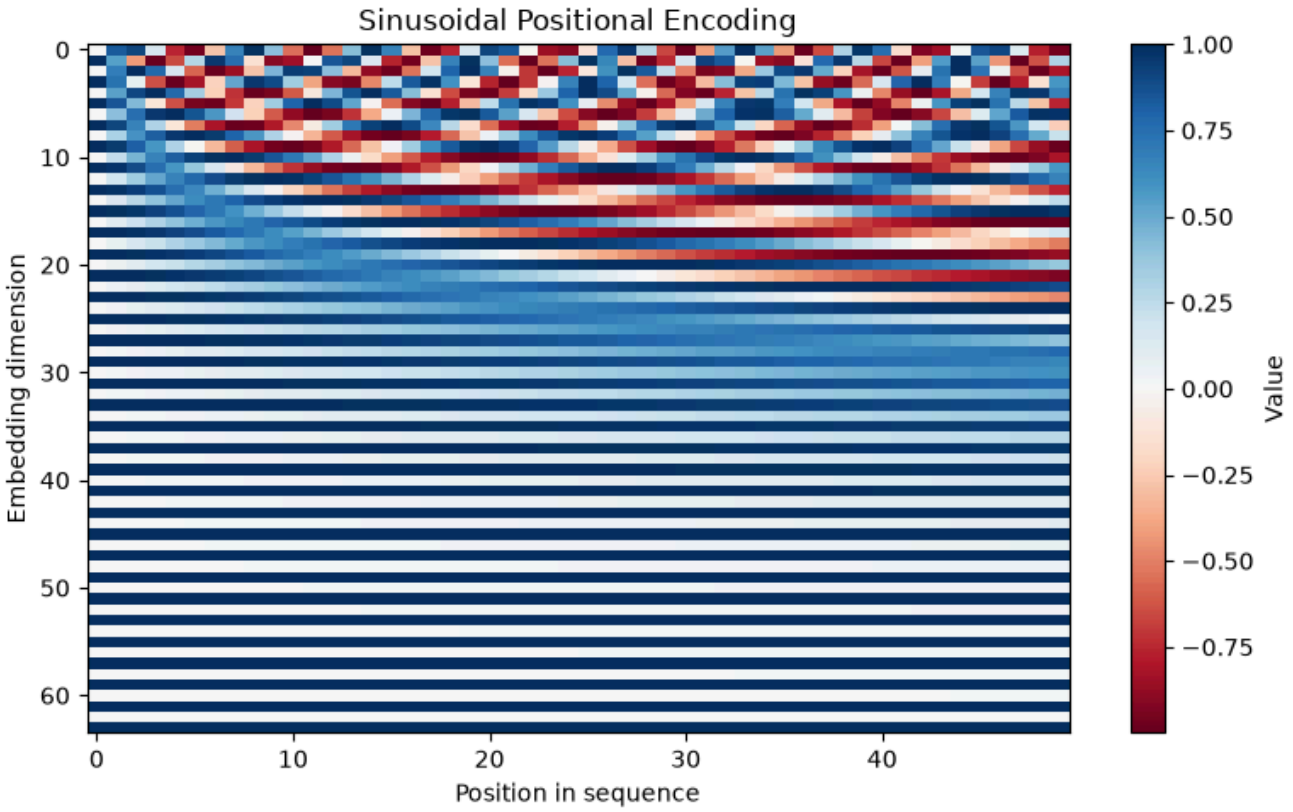
Plot it as a heatmap, transposed so position runs along the x-axis and dimension down the y-axis.

PYTHON

`pe.T` transposes. `aspect="auto"` lets the image stretch to fill the axes instead of forcing square pixels. `cmap="RdBu"` is a diverging map — appropriate here because the values are signed, running −1 to +1 with white at zero.

OUTPUT

```
Positional encoding shape: (50, 64)
```



Each **column** is one position's unique signature. Each **row** is one dimension oscillating at its own frequency — fast at the top, slow at the bottom, which is why the stripes stretch out as you move down.

WHY SINUSOIDS RATHER THAN JUST NUMBERING THE POSITIONS?

- **Bounded.** Every value stays in $[-1, 1]$, so adding it cannot swamp the embedding. Raw integers would grow without limit.
- **Unique.** No two positions share a signature within any practical length.
- **Relative offsets are linear.** $PE(pos + k)$ is a fixed rotation of $PE(pos)$ — the same rotation regardless of pos . A linear layer can therefore learn "attend three tokens back". Section 12 verifies this numerically.

- **Extrapolates.** Nothing is learned, so the formula gives a code for position 5000 even if training never saw a sequence that long.

12 Exercises — positional encoding shapes and uniqueness

Four exercises here: two about shapes, and two about whether positions can collide. The second pair is where the interesting mathematics lives.

Shapes: `seq_len` 50 → 10, and `d_model` 64 → 8

Solution

```
# Exercise - change seq_len from 50 to 10. What is the new shape?

pe_10 = positional_encoding(seq_len=10, d_model=64)
print("seq_len=10, d_model=64 ->", pe_10.shape)      # (10, 64)

# Exercise - change d_model from 64 to 8. What is the new shape?

pe_d8 = positional_encoding(seq_len=50, d_model=8)
print("seq_len=50, d_model=8  ->", pe_d8.shape)      # (50, 8)
pe_small = positional_encoding(seq_len=10, d_model=8)
print("both changed           ->", pe_small.shape)   # (10, 8)

print("""
The shape is always (seq_len, d_model), and the two arguments are independent:
  seq_len  = how many positions you need a code for   -> number of ROWS
  d_model  = the width of the model's vectors         -> number of COLUMNS

d_model must match the token embeddings, because the two are ADDED elementwise
(x = token_embeddings + pe). seq_len just has to cover the longest sentence.""")

# The rows do not depend on seq_len at all - position 3's code is the same
# whether you asked for 10 positions or 50:
print("row 3 identical in both:", np.allclose(pe_10[3], positional_encoding(50, 64)[

# ...but they DO depend on d_model, because d_model sets the frequencies:
print("row 3 same across widths:", np.allclose(pe_small[3][:8], pe_10[3][:8]))
print("  -> False: the exponent 2i/d_model changes when d_model changes.")
```

L3–4	Fewer positions, same width → `(10, 64)`.
L8–12	Same positions, narrower model → `(50, 8)`; both changed → `(10, 8)`. PYTHON Passing arguments by name (`seq_len=10, d_model=64`) rather than by position makes which is which unambiguous — worth the extra characters.
L14–21	What the two arguments actually control, and the constraint that matters: `d_model` must match the token embeddings, because the two arrays are *added*.
L23–24	A check worth making: position 3's code does **not** depend on how many positions you asked for. The formula is a pure function of `(pos, i, d_model)`.
L26–28	But it *does* depend on `d_model`, because `d_model` appears in the exponent $2i/d$ and so sets the frequencies.

OUTPUT

```
seq_len=10, d_model=64 -> (10, 64)
seq_len=50, d_model=8  -> (50, 8)
both changed           -> (10, 8)

The shape is always (seq_len, d_model), and the two arguments are independent:
  seq_len  = how many positions you need a code for   -> number of ROWS
  d_model  = the width of the model's vectors         -> number of COLUMNS

d_model must match the token embeddings, because the two are ADDED elementwise
(x = token_embeddings + pe). seq_len just has to cover the longest sentence.
row 3 identical in both: True
row 3 same across widths: False
  -> False: the exponent 2i/d_model changes when d_model changes.
```

Are positions 0, 1 and 2 identical?

The task: inspect `pe[0]`, `pe[1]`, `pe[2]`. Are they identical? Why?

Solution

```python
# Exercise - are the encodings for positions 0, 1 and 2 identical? Why?
#
# Short answer: no, and they cannot be.

print("pe[0][:8] =", np.round(pe[0][:8], 4))
print("pe[1][:8] =", np.round(pe[1][:8], 4))
print("pe[2][:8] =", np.round(pe[2][:8], 4))
print("\npe[0] == pe[1]?", np.allclose(pe[0], pe[1]))
print("pe[1] == pe[2]?", np.allclose(pe[1], pe[2]))

# Position 0 is the one special-looking row: sin(0)=0 and cos(0)=1, so it is
# exactly [0, 1, 0, 1, ...] regardless of d_model.
print("\npe[0] is exactly [0,1,0,1,...]:",
      np.allclose(pe[0][0::2], 0) and np.allclose(pe[0][1::2], 1))

# Every DIMENSION PAIR has its own frequency:  omega_i = 1 / 10000^(2i/d)
d_model = pe.shape[1]
i = np.arange(0, d_model, 2)
omega = 1.0 / (10000 ** (i / d_model))
print("\nfastest pair omega =", round(omega[0], 6),
      " period =", round(2 * np.pi / omega[0], 2), "positions")
print("slowest pair omega =", round(omega[-1], 8),
      " period =", round(2 * np.pi / omega[-1], 1), "positions")
print("""
Two positions could only collide if EVERY one of those sine/cosine pairs
agreed at once. The slowest pair has a period of tens of thousands of
positions, so within any real sequence length no two positions repeat.
That is the whole point: the code has to be a unique fingerprint per slot.""")

# The deeper property: shifting position by k is a fixed ROTATION, the same
# one for every starting position. That is what makes RELATIVE offsets
# learnable by a linear layer.
def rot(theta):
    return np.array([[np.cos(theta), np.sin(theta)],
                     [-np.sin(theta), np.cos(theta)]])

k, pair = 5, 3                      # shift by 5 positions, look at pair 3
w = omega[pair]
v_p = np.array([pe[7, 2 * pair], pe[7, 2 * pair + 1]])       # PE(7)  for that pair
v_q = np.array([pe[7 + k, 2 * pair], pe[7 + k, 2 * pair + 1]])  # PE(12) for that pa
print("PE(12) reached by rotating PE(7):", np.allclose(rot(w * k) @ v_p, v_q))

# ...and the same rotation works from any starting position:
ok = all(np.allclose(rot(w * k) @ np.array([pe[p, 2*pair], pe[p, 2*pair+1]]),
                     np.array([pe[p+k, 2*pair], pe[p+k, 2*pair+1]]))
```

```
          for p in range(0, 40))
print("same rotation works from every position p:", ok)

# Similarity between position 0 and every other position decays smoothly -
# nearby positions have similar codes, distant ones do not.
sims = pe @ pe[0] / (np.linalg.norm(pe, axis=1) * np.linalg.norm(pe[0]))
fig, ax = plt.subplots(figsize=(7, 3.2))
ax.plot(sims, color="#0B6E63", lw=2)
ax.set_xlabel("Position")
ax.set_ylabel("Cosine similarity to position 0")
ax.set_title("Positional codes fade smoothly with distance")
ax.grid(alpha=0.3)
plt.tight_layout()
plt.savefig("pe_similarity.png", dpi=120)
plt.show()
print("\ncosine similarity to position 0, first 6 positions:", np.round(sims[:6], 3))
```

L5–11

Print the first eight entries of each and check for equality. They are not identical.

PYTHON

`pe[0][:8]` is two lookups: row 0, then its first eight entries. `np.allclose` rather than `==`, because these are floats.

L13–16

Position 0 *is* the odd-looking one: $\sin 0 = 0$ and $\cos 0 = 1$, so its code is exactly `[0,1,0,1,...]` for any `d_model`. Every other position is a genuine mixture.

PYTHON

`pe[0][0::2]` takes the even-indexed entries, `[1::2]` the odd ones. Two `np.allclose` checks joined with `and`.

L18–25

Recover the frequencies from `d_model` and report the fastest and slowest periods.

PYTHON

`10000 ** (i / d_model)` raises elementwise across the whole array. Period is $2\pi/\omega$.

L26–31

The uniqueness argument: a collision requires *every* sine/cosine pair to agree simultaneously, and the slowest pair has a period of ~47,000 positions.

L33–48

The rotation property, verified numerically. For one dimension pair, shifting position by *k* is exactly multiplication by a fixed 2×2 rotation matrix — and the same matrix works from every starting position.

PYTHON

`rot(theta)` builds the rotation matrix. The `all(... for p in range(0, 40))` is a generator inside `all()`: true only if the check passes at every starting position.

L50–61

Plot cosine similarity between position 0 and every other position, to show the codes fade smoothly with distance rather than jumping around.

PYTHON

`np.linalg.norm(pe, axis=1)` gives each row's length; dividing the dot products by the product of lengths turns them into cosines.

OUTPUT

```
pe[0][:8] = [0. 1. 0. 1. 0. 1. 0. 1.]
pe[1][:8] = [0.8415 0.5403 0.6816 0.7318 0.5332 0.846  0.4093 0.9124]
pe[2][:8] = [ 0.9093 -0.4161  0.9975  0.0709  0.9021  0.4315  0.7469  0.6649]

pe[0] == pe[1]? False
pe[1] == pe[2]? False

pe[0] is exactly [0,1,0,1,...]: True

fastest pair omega = 1.0  period = 6.28 positions
slowest pair omega = 0.00013335  period = 47117.2 positions

Two positions could only collide if EVERY one of those sine/cosine pairs
agreed at once. The slowest pair has a period of tens of thousands of
positions, so within any real sequence length no two positions repeat.
That is the whole point: the code has to be a unique fingerprint per slot.
PE(12) reached by rotating PE(7): True
same rotation works from every position p: True

cosine similarity to position 0, first 6 positions: [1.    0.966 0.884 0.8   0.748 0.734]
```

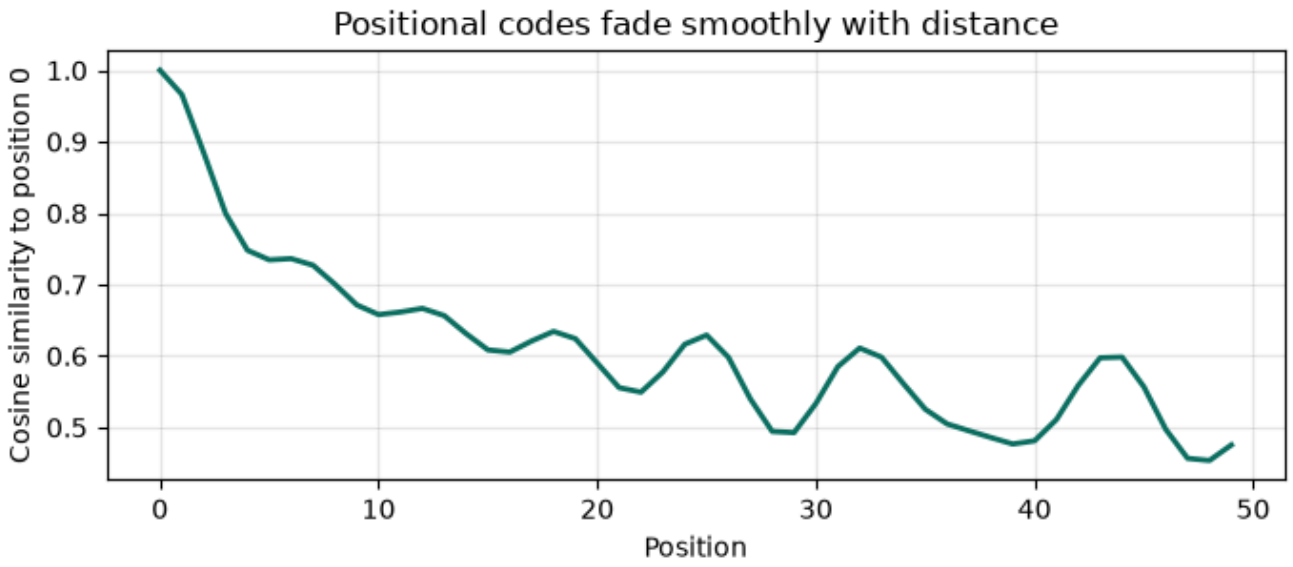
THE ROTATION IDENTITY — WHY THIS ENCODING WAS CHOSEN

For the dimension pair with frequency ω , the code is the point $(\sin \omega p,\ \cos \omega p)$ on a circle. Shifting the position by k rotates that point by a fixed angle ωk :

$$\begin{bmatrix} \sin \omega(p+k) \\ \cos \omega(p+k) \end{bmatrix} = \begin{bmatrix} \cos \omega k & \sin \omega k \\ -\sin \omega k & \cos \omega k \end{bmatrix} \begin{bmatrix} \sin \omega p \\ \cos \omega p \end{bmatrix} \qquad \text{rotation}$$

This is just the angle-addition formulas $\sin(a+b) = \sin a \cos b + \cos a \sin b$ and $\cos(a+b) = \cos a \cos b - \sin a \sin b$ written as a matrix.

The consequence is the point: $R(\omega k)$ **depends only on** k **, not on** p . So a single linear transformation can express “look k positions back” everywhere in the sequence at once. The code verifies exactly this — the same rotation works from all 40 tested starting positions. That is why sinusoids, rather than any other unique fingerprint.



Cosine similarity between position 0 and each later position. Nearby positions have similar codes and distant ones do not — a smooth, learnable notion of “near”.

ANSWER, IN ONE LINE

No, they are not identical, and no two positions ever are within a practical sequence length. Position 0 is exactly `[0,1,0,1,...]`; every other position is a unique mixture of frequencies; and consecutive positions are related by a fixed rotation, which is what makes *relative* position learnable.

13 The complete encoder block

Everything assembled: **input + positional encoding → multi-head self-attention → add & norm → feed-forward → add & norm**. This is the brick that GPT-3 stacks ninety-six times.

THE THREE PIECES YOU HAVE NOT SEEN YET

Residual connection. Instead of $x \leftarrow f(x)$, compute:

$$x \leftarrow x + f(x)$$

The gradient of $x + f(x)$ with respect to x is $I + f'(x)$ — that I is a path the gradient can always take, even if f' contributes nothing. Without it, deep stacks do not train.

Layer normalisation. Standardise each token's vector across its features:

$$\mathrm{LayerNorm}(x) = \frac{x-\mu}{\sigma+\epsilon}, \qquad \mu = \frac{1}{d}\sum_{j=1}^{d} x_j, \quad \sigma = \sqrt{\frac{1}{d}\sum_{j=1}^{d}(x_j-\mu)^2}$$

Note this is per *token*, across features — not across the batch. That is why it works with any sequence length and does not care what else is in the batch.

Feed-forward network. Attention only *mixes* vectors that already exist; it cannot compute a new nonlinear function of a token. The FFN does that, applied to each position independently:

$$\mathrm{FFN}(x) = \max(0,\ xW_1 + b_1)\,W_2 + b_2$$

Widening to d_{ff} and back is where most of a transformer's parameters actually live — typically $d_{ff} = 4d_{\text{model}}$.

Cell 6 — the full encoder block

```
def layer_norm(x, eps=1e-6):
    mean = x.mean(axis=-1, keepdims=True)
    std = x.std(axis=-1, keepdims=True)
    return (x - mean) / (std + eps)

def feed_forward(x, d_ff=16, seed=0):
    rng = np.random.RandomState(seed)
    W1 = rng.randn(x.shape[-1], d_ff) * 0.3
    W2 = rng.randn(d_ff, x.shape[-1]) * 0.3
    hidden = np.maximum(0, x @ W1)  # ReLU
    return hidden @ W2

def encoder_block(token_embeddings, num_heads=4, seed=0):
    seq_len, d_model = token_embeddings.shape

    # Step 1: add positional encoding
    pe = positional_encoding(seq_len, d_model)
    x = token_embeddings + pe

    # Step 2: multi-head self-attention, then residual add + layer norm
    attn_out, _ = multi_head_attention(x, num_heads=num_heads, d_model=d_model, seed
    x = layer_norm(x + attn_out)

    # Step 3: feed-forward network, then residual add + layer norm
    ff_out = feed_forward(x, seed=seed + 1)
    x = layer_norm(x + ff_out)

    return x

encoder_output = encoder_block(X, num_heads=4, seed=2)
print("Input shape:  ", X.shape)
print("Output shape: ", encoder_output.shape)
print("\nOutput is the same shape as the input -- this is what lets encoder blocks s
```

L1–4

Layer norm. Subtract the row mean, divide by the row standard deviation.

PYTHON

Both reductions use `axis=-1, keepdims=True` so each token is normalised against its own statistics and broadcasting lines up. `eps=1e-6` is scientific notation for 0.000001.

L4

The `+ eps` prevents division by zero when a token's features are all equal ($\sigma = 0$). Note this implementation has no learnable γ and β — a real `LayerNorm` adds them so the network can undo the normalisation where it needs to.

L6–7

The feed-forward network. `d_ff=16` is twice `d_model=8` here; real models use 4×.

L8–9

Two weight matrices: `(d_model, d_ff)` up, then `(d_ff, d_model)` back down. No biases in this simplified version.

PYTHON

`x.shape[-1]` reads the width off the input, so the function adapts to whatever it is given.

L10

Expand, then ReLU. This nonlinearity is the entire reason the FFN exists — without it, two stacked linear layers would collapse into one.

PYTHON

`np.maximum(0, z)` takes the elementwise maximum against zero, so negatives become 0 and positives pass through. Do not confuse it with `np.max`, which reduces an array to a single value.

L11

Project back down to `d_model`, so the residual addition is well defined.

L13–14

The block itself. Read the shape off the input rather than taking it as an argument.

PYTHON

`seq_len, d_model = token_embeddings.shape` unpacks the two-element shape tuple into two names.

L16–18

Step 1 — add positional encoding, so the tokens stop being an unordered set.

PYTHON

Two arrays of identical shape added elementwise. This is the only place order enters the model.

L20–22

Step 2 — self-attention, then *add the input back* and normalise. The `x + attn_out` is the residual; the `layer_norm(...)` around it keeps the scale stable.

PYTHON

`attn_out, _ = ...` keeps the output and discards the weights. This block uses the post-norm arrangement of the original paper; most modern models use pre-norm, which trains more stably at depth.

L24–26

Step 3 — feed-forward, residual, norm again. Same pattern as step 2.

PYTHON

`seed + 1` gives the FFN different random weights from the attention projections — otherwise they would be identical, which would be a strange coincidence to build in.

L28

Return the transformed embeddings, same shape as the input.

L30–34

Run it and confirm the shape survives.

OUTPUT

```
Input shape:   (10, 8)
Output shape:  (10, 8)

Output is the same shape as the input -- this is what lets encoder blocks stack N times.
```

THE INVARIANT THAT MAKES DEPTH POSSIBLE

Input `(10, 8)`, output `(10, 8)`. Every sublayer must preserve shape, because the residual computes $x + f(x)$ and that is only defined when $f(x)$ matches x. This is what forces attention to project back to d_{model} and the FFN to return from d_{ff}. Get it right once, and the block becomes a component you can stack any number of times.

14

Exercise — layer norm, mean and standard deviation

The task: pass an array to `layer_norm`, then compute the mean and standard deviation of the result.

Solution

```python
# Exercise - layer_norm, then check the mean and standard deviation.

x = np.array([[1, 2, 3, 4],
              [10, 20, 30, 40]], dtype=float)
y = layer_norm(x)
print(y)
print("mean per row:", y.mean(axis=-1))
print("std  per row:", y.std(axis=-1))

# The two rows come out IDENTICAL, even though the inputs differ by 10x:
print("\nrow 0 == row 1 after norm:", np.allclose(y[0], y[1]))
print("""
That is the whole point of layer norm. It subtracts the row's mean and divides
by the row's standard deviation, so any input of the form a*x + b (with a > 0)
lands on exactly the same output. The layer keeps the SHAPE of a token's
feature pattern and throws away its overall scale and offset.""")

a, b = 7.0, -3.0
print("layer_norm(7x - 3) == layer_norm(x):", np.allclose(layer_norm(a * x + b), y))

# The std is 0.99999... rather than exactly 1 because of the eps on the
# denominator, which exists to stop a division by zero on a constant row:
print("\nstd is off by:", 1 - y.std(axis=-1))
flat = np.array([[5.0, 5.0, 5.0, 5.0]])
print("a constant row survives without dividing by zero:", layer_norm(flat))

# Note this implementation normalises with the POPULATION std (ddof=0), and has
# no learnable gain/bias. A real nn.LayerNorm adds  gamma * xhat + beta  so the
# network can undo the normalisation where it needs to.
print("numpy default ddof:", np.std([1., 2., 3., 4.]), "(population, = ddof 0)")
```

L3–9

Two rows that differ by a factor of 10, normalised, then measured. Mean 0 and standard deviation 1 per row, as designed.

L11–12

The striking part. The two output rows are *identical*, despite inputs of `[1,2,3,4]` and `[10,20,30,40]`.

L13–18

Why: layer norm is invariant to any positive affine rescaling of a row. It keeps the *shape* of a token's feature pattern and throws away scale and offset entirely.

L20–21

The invariance stated as a check: $\mathrm{LN}(ax+b) = \mathrm{LN}(x)$ for $a > 0$.

> PYTHON
> Multiplying and adding scalars to an array applies elementwise, so `7 * x - 3` transforms every entry at once.

L23–27

Two loose ends. The standard deviation comes out as 0.9999991 rather than 1 because of the `eps` in the denominator; and a constant row survives instead of dividing by zero, which is what `eps` is for.

> PYTHON
> `1 - y.std(axis=-1)` shows the tiny discrepancy directly rather than making you compare long decimals by eye.

L29–31

A caveat: NumPy's `std` defaults to the population formula (dividing by d , not $d-1$), and this implementation has no learnable gain or bias.

OUTPUT

```
[[-1.34163959 -0.4472132   0.4472132   1.34163959]
 [-1.34164067 -0.44721356  0.44721356  1.34164067]]
mean per row: [0. 0.]
std  per row: [0.99999911 0.99999991]

row 0 == row 1 after norm: True

That is the whole point of layer norm. It subtracts the row's mean and divides
by the row's standard deviation, so any input of the form a*x + b (with a > 0)
lands on exactly the same output. The layer keeps the SHAPE of a token's
feature pattern and throws away its overall scale and offset.
layer_norm(7x - 3) == layer_norm(x): True

std is off by: [8.94426391e-07 8.94427111e-08]
a constant row survives without dividing by zero: [[0. 0. 0. 0.]]
numpy default ddof: 1.118033988749895 (population, = ddof 0)
```

WHY IDENTICAL OUTPUTS ARE THE CORRECT BEHAVIOUR

Substituting $x' = ax + b$ into the definition, the mean becomes $a\mu + b$ and the standard deviation becomes $a\sigma$ (for $a > 0$), so:

$$\mathrm{LN}(ax+b) = \frac{(ax+b)-(a\mu+b)}{a\sigma} = \frac{a(x-\mu)}{a\sigma} = \frac{x-\mu}{\sigma} = \mathrm{LN}(x)$$

The a cancels and the b subtracts away. In a transformer this is exactly what you want: after a residual addition, activations can drift to any scale, and layer norm hands the next sublayer a consistently scaled input every time — which is what stops deep stacks from exploding or vanishing.

15 Final Exercise 1 — Scaling matters

The task: remove the `/ np.sqrt(d_k)` scaling, rerun with a much larger `d_model` (pad the features to 256 with zeros), and compare. Do the weights become peaky? Why does that hurt learning?

THE SUGGESTED PROCEDURE DOES NOT SHOW THE EFFECT — AND THAT IS WORTH KNOWING

Zero-padding cannot change `Q @ K.T`. A zero column contributes $0 \times 0 = 0$ to every dot product, so the score matrix at $d_k = 256$ is **identical** to the one at $d_k = 8$. The only thing that changes is the divisor.

So padding makes the *scaled* version flatter (dividing by 16 instead of 2.83), and removing the scaling just hands back the original 8-dimensional scores. The effect the exercise is pointing at is real, but demonstrating it needs vectors that carry signal in every dimension — which is what a trained model has. Part A below does it as written; Part B does it properly.

Solution

```python
# ============================================================================
# Section 8, Exercise 1 - Scaling matters
# ============================================================================
# Claim to test: without the 1/sqrt(d_k) divisor, attention weights get
# "peaky" (nearly one-hot) as d_k grows, and that kills the gradient.

def attention_unscaled(Q, K, V):
    scores = Q @ K.T                        # note: no / sqrt(d_k)
    weights = softmax(scores, axis=-1)
    return weights @ V, weights

# --- Part A: the exercise as literally written (pad the features to 256) -----
X_pad = np.zeros((X.shape[0], 256))
X_pad[:, :X.shape[1]] = X                   # same features, 248 zeros bolted on

_, w_scaled_8 = scaled_dot_product_attention(X, X, X)
_, w_scaled_256 = scaled_dot_product_attention(X_pad, X_pad, X_pad)
_, w_unscaled_256 = attention_unscaled(X_pad, X_pad, X_pad)

print("PART A - zero-padding the hand-made features to d_k=256")
print("  max weight, scaled   d_k=8  :", round(w_scaled_8[it_idx].max(), 4))
print("  max weight, scaled   d_k=256:", round(w_scaled_256[it_idx].max(), 4))
print("  max weight, unscaled d_k=256:", round(w_unscaled_256[it_idx].max(), 4))
print("  unscaled-256 equals scaled-8?",
      np.allclose(w_unscaled_256, softmax(X @ X.T, axis=-1)))
print("""
  Padding with ZEROS does not do what the exercise implies. Q @ K.T is
  completely unchanged by zero columns, so the only thing that moved is the
  divisor: sqrt(256)=16 instead of sqrt(8)=2.83. The scaled version therefore
  gets FLATTER, not peakier, and dropping the scaling just gives back the
  d_k=8 scores exactly.

  The peakiness effect is real, but it needs vectors whose entries actually
  carry signal in every dimension - which is what a trained model has.""")

# --- Part B: the effect the exercise is pointing at --------------------------
# For q, k with independent entries of mean 0 and variance 1:
#     q.k = sum of d_k independent terms  ->  Var(q.k) = d_k,  std = sqrt(d_k)
# So the logits GROW like sqrt(d_k), and softmax saturates.
#
# "mean p(1-p)" below is a proxy for gradient health: the softmax Jacobian
# carries that factor, so when it collapses, so does learning.

def report(scale):
    rng = np.random.RandomState(0)
```

```
    print(f"{'d_k':>6s}{'std of scores':>16s}{'max weight':>13s}"
          f"{'entropy':>10s}{'mean p(1-p)':>14s}")
    print("-" * 59)
    for d_k in (8, 64, 256, 1024):
        Qr, Kr = rng.randn(6, d_k), rng.randn(6, d_k)
        s = Qr @ Kr.T
        if scale:
            s = s / np.sqrt(d_k)
        w = softmax(s, axis=-1)
        ent = -(w * np.log(w + 1e-12)).sum(axis=-1).mean()
        print(f"{d_k:>6d}{s.std():>16.2f}{w.max():>13.4f}{ent:>10.3f}"
              f"{(w * (1 - w)).mean():>14.2e}")

print("\nPART B - random vectors, the realistic case")
print("\nWITHOUT the 1/sqrt(d_k) scaling:")
report(scale=False)
print("\nWITH the scaling:")
report(scale=True)

print("""
Unscaled, the score spread grows exactly like sqrt(d_k) (2.6 -> 29 as d_k goes
8 -> 1024), the largest weight saturates at 1.0000, and entropy collapses from
0.80 to 0.02: the softmax has quietly become a hard argmax.

Why that hurts learning. The softmax Jacobian is

    d p_i / d z_j  =  p_i (delta_ij - p_j)

so every gradient term carries a factor of p(1-p). Saturated p means that factor
goes to zero: the mean falls from 6.6e-02 to 1.5e-03, a factor of ~44. Almost no
gradient reaches the queries and keys, and the layer stops learning.

With the scaling, all four columns are essentially flat - std ~1, entropy ~1.5,
gradient scale ~0.12 - regardless of width. That is the entire job of the
divisor: hold the score variance at 1 so the softmax stays in the region where
it still has a derivative, no matter how wide the model gets.""")
```

L6–10	Attention with the scaling removed — otherwise character-for-character the original.
L13–15	Build the padded input: same 8 features, then 248 zero columns. PYTHON `np.zeros((10, 256))` makes the array, then `X_pad[:, :8] = X` writes the real features into the first eight columns. `:8` means “up to but not including index 8”.
L17–19	Three attention runs to compare: scaled at 8 dimensions, scaled at 256, unscaled at 256.
L21–26	The numbers, and the check that proves the point: unscaled-256 is exactly equal to what plain unscaled 8-dimensional scores would give. PYTHON `np.allclose(w_unscaled_256, softmax(X @ X.T, axis=-1))` compares the full matrices, not just one entry.
L27–38	The explanation of why Part A behaves as it does.
L41–47	Part B's setup. The variance argument from Section 2, restated as a comment so the table has context.
L48–61	A reusable reporting function so scaled and unscaled runs are measured identically — same seed, same data, one flag different. PYTHON Defining a function to run both cases removes the risk of accidentally comparing different random draws. `np.random.RandomState(0)` is re-created inside, so both calls see the identical sequence.
L52–59	Four metrics per width: the spread of the scores, the largest single weight, the entropy of the distribution, and the mean of $p(1-p)$ as a proxy for gradient health. PYTHON Entropy is `-(w * np.log(w + 1e-12)).sum(axis=-1).mean()`. The `1e-12` prevents `log(0)`, which would be `-inf`.
L63–67	Run both tables.
L69–81	The conclusion, tied to the specific numbers above it.

OUTPUT

```
PART A - zero-padding the hand-made features to d_k=256
  max weight, scaled   d_k=8  : 0.1625
  max weight, scaled   d_k=256: 0.1097
  max weight, unscaled d_k=256: 0.3016
  unscaled-256 equals scaled-8? True

  Padding with ZEROS does not do what the exercise implies. Q @ K.T is
  completely unchanged by zero columns, so the only thing that moved is the
  divisor: sqrt(256)=16 instead of sqrt(8)=2.83. The scaled version therefore
  gets FLATTER, not peakier, and dropping the scaling just gives back the
  d_k=8 scores exactly.

  The peakiness effect is real, but it needs vectors whose entries actually
  carry signal in every dimension - which is what a trained model has.

PART B - random vectors, the realistic case

WITHOUT the 1/sqrt(d_k) scaling:
   d_k   std of scores   max weight   entropy   mean p(1-p)
-------------------------------------------------------------
     8            2.62       0.8453     0.799      6.64e-02
    64            8.72       0.9836     0.457      4.32e-02
   256           15.81       1.0000     0.030      1.92e-03
  1024           29.00       1.0000     0.024      1.49e-03

WITH the scaling:
   d_k   std of scores   max weight   entropy   mean p(1-p)
-------------------------------------------------------------
     8            0.93       0.4623     1.539      1.24e-01
    64            1.09       0.5149     1.490      1.22e-01
   256            0.99       0.5854     1.459      1.19e-01
  1024            0.91       0.4657     1.580      1.26e-01

Unscaled, the score spread grows exactly like sqrt(d_k) (2.6 -> 29 as d_k goes
8 -> 1024), the largest weight saturates at 1.0000, and entropy collapses from
0.80 to 0.02: the softmax has quietly become a hard argmax.

Why that hurts learning. The softmax Jacobian is

    d p_i / d z_j  =  p_i (delta_ij - p_j)

so every gradient term carries a factor of p(1-p). Saturated p means that factor
goes to zero: the mean falls from 6.6e-02 to 1.5e-03, a factor of ~44. Almost no
gradient reaches the queries and keys, and the layer stops learning.

With the scaling, all four columns are essentially flat - std ~1, entropy ~1.5,
gradient scale ~0.12 - regardless of width. That is the entire job of the
divisor: hold the score variance at 1 so the softmax stays in the region where
it still has a derivative, no matter how wide the model gets.
```

What the tables show

D_K	SCORE SPREAD	MAX WEIGHT	ENTROPY	GRADIENT SCALE
8	2.62	0.845	0.799	6.6e−02
64	8.72	0.984	0.457	4.3e−02
256	15.81	1.0000	0.030	1.9e−03
1024	**29.00**	**1.0000**	**0.024**	**1.5e−03**

Without scaling. The spread grows exactly as $\sqrt{d_k}$ predicts (√8 ≈ 2.8 up to √1024 = 32), the top weight saturates at 1.0, entropy collapses to near zero, and the gradient scale falls by a factor of ~44.

With scaling, every column is flat regardless of width: spread ~1, entropy ~1.5, gradient scale ~0.12. That is the whole job of the divisor.

WHY PEAKY ATTENTION KILLS LEARNING

The softmax Jacobian is

$$\frac{\partial p_i}{\partial z_j} = p_i(\delta_{ij} - p_j)$$

Every entry carries a factor of p . When one weight saturates at 1 and the rest at 0, that factor goes to zero, and the chain rule multiplies the upstream gradient by ~0. Concretely, the gradient reaching the queries is

$$\frac{\partial L}{\partial q_i} = \frac{1}{\sqrt{d_k}} \sum_j \frac{\partial L}{\partial p_{ij}} p_{ij} \left(k_j - \sum_l p_{il} k_l \right)$$

If p_{ij} is one-hot, the bracket becomes $k_j - k_j = 0$ and the whole sum vanishes. The attention pattern freezes at whatever the random initialisation happened to pick, and no amount of training moves it. Dividing by $\sqrt{d_k}$ keeps p in the soft region where the derivative is largest — $p(1-p)$ is maximised at $p = 0.5$.

16 Final Exercise 2 — Masked self-attention

The task: write `causal_mask(seq_len)` returning a matrix of `0`s and `-inf`s such that adding it to the scores before the softmax stops any position attending to the future. Verify position 0 attends only to itself, position 1 to 0–1, and so on.

This is the decoder's “masked multi-head self-attention”, and it is the single change that separates BERT-style encoders from GPT-style generators.

Solution

```python
# ============================================================================
# Section 8, Exercise 2 - Masked (causal) self-attention
# ============================================================================
# A decoder generates left to right, so position t must not see t+1, t+2, ...
# The trick: add -inf to the forbidden scores BEFORE the softmax. Since
# exp(-inf) = 0, those positions get exactly zero weight, and the remaining
# weights still sum to 1 on their own.

def causal_mask(seq_len):
    """Upper triangle (strictly above the diagonal) = -inf, everything else 0."""
    return np.triu(np.full((seq_len, seq_len), -np.inf), k=1)

print("causal_mask(5):")
print(causal_mask(5))

def masked_attention(Q, K, V):
    d_k = Q.shape[-1]
    scores = Q @ K.T / np.sqrt(d_k)
    scores = scores + causal_mask(Q.shape[0])   # the only added line
    weights = softmax(scores, axis=-1)
    return weights @ V, weights

_, w_causal = masked_attention(X, X, X)
print("\nmasked attention weights (rounded):")
print(np.round(w_causal, 3))

# --- verification ------------------------------------------------------
print("\nposition 0 attends only to itself:", np.allclose(w_causal[0], [1] + [0] * 9
ok_zero = all(np.allclose(w_causal[i, i + 1:], 0) for i in range(len(X)))
ok_sum = np.allclose(w_causal.sum(axis=1), 1.0)
print("every row has zero weight on the future:", ok_zero)
print("every row still sums to 1:", ok_sum)
for i in (0, 1, 2, 9):
    nz = int((w_causal[i] > 0).sum())
    print(f"  row {i}: {nz} non-zero weight(s) -> positions 0..{i}")

# Why -inf and not just zeroing the weights afterwards: zeroing after the
# softmax would leave the row summing to less than 1, so the token's output
# would silently shrink. Masking BEFORE means the surviving positions
# renormalise among themselves.
scores_plain = X @ X.T / np.sqrt(X.shape[-1])
after = softmax(scores_plain, axis=-1) * (causal_mask(len(X)) == 0)
print("\nnaive 'zero it out afterwards' row sums:", np.round(after.sum(axis=1), 3))
print("-> not 1. Masking before the softmax is what keeps it a distribution.")

```

```
fig, axes = plt.subplots(1, 2, figsize=(11, 4.4))
for ax, w, title in ((axes[0], attn_weights, "Encoder: full self-attention"),
                     (axes[1], w_causal, "Decoder: causal mask")):
    im = ax.imshow(w, cmap="viridis", vmin=0, vmax=w.max())
    ax.set_xticks(range(len(tokens_list))); ax.set_yticks(range(len(tokens_list)))
    ax.set_xticklabels(tokens_list, rotation=45, ha="right", fontsize=8)
    ax.set_yticklabels(tokens_list, fontsize=8)
    ax.set_title(title)
    fig.colorbar(im, ax=ax)
plt.tight_layout()
plt.savefig("causal_mask.png", dpi=120)
plt.show()
```

L1–8

The idea: add $-\infty$ to forbidden scores *before* the softmax. Since $e^{-\infty} = 0$, those positions get exactly zero weight, and the survivors renormalise among themselves.

L10–12

The mask itself, in one line.

> PYTHON
>
> `np.full((n,n), -np.inf)` makes an n×n array of −infinity. `np.triu(..., k=1)` keeps the upper triangle *strictly above* the diagonal and zeroes the rest — `k=1` is what excludes the diagonal, so a token can still attend to itself.

L14–15

Print it, because a mask is far easier to check by eye than to reason about.

L17–22

Attention with one line added. Everything else is unchanged from Section 3 — masking is genuinely this small a modification.

> PYTHON
>
> `scores + causal_mask(...)` adds elementwise. Adding −inf to a finite number gives −inf; adding 0 leaves it alone.

L24–26

Run it on the ten-token sentence.

L29–38

The verification the exercise asks for: row 0 is exactly `[1,0,0,...]`, no row has weight on the future, and every row still sums to 1.

> PYTHON
>
> `all(... for i in range(len(X)))` checks every row in one expression. `w_causal[i, i+1:]` slices out everything to the right of the diagonal on row *i*.

L40–48

Why masking must happen before the softmax. Zeroing the weights afterwards leaves each row summing to less than 1 — 0.117 for row 0 — so the token's output silently shrinks toward zero. Masking first lets the remaining positions renormalise.

> PYTHON
>
> `(causal_mask(n) == 0)` gives a boolean array, and multiplying by it zeroes the masked entries — the naive approach, shown here precisely so you can see it fail.

L50–60

Plot the two matrices side by side. The triangular structure is the entire difference between an encoder and a decoder.

> PYTHON
>
> `plt.subplots(1, 2)` returns an array of two axes, unpacked as `axes[0]` and `axes[1]`. Looping over a tuple of `(axis, data, title)` triples avoids writing the plotting code twice.

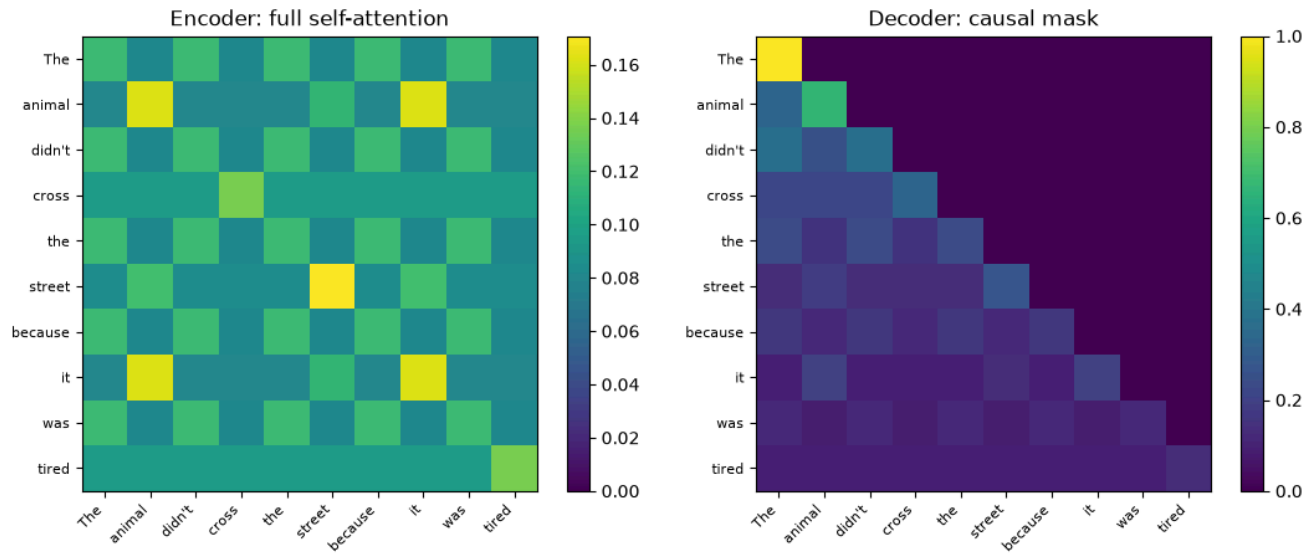
OUTPUT

```
causal_mask(5):
[[  0. -inf -inf -inf -inf]
 [  0.   0. -inf -inf -inf]
 [  0.   0.   0. -inf -inf]
 [  0.   0.   0.   0. -inf]
 [  0.   0.   0.   0.   0.]]

masked attention weights (rounded):
[[1.    0.    0.    0.    0.    0.    0.    0.    0.    0.   ]
 [0.33  0.67  0.    0.    0.    0.    0.    0.    0.    0.   ]
 [0.37  0.26  0.37  0.    0.    0.    0.    0.    0.    0.   ]
 [0.226 0.226 0.226 0.322 0.    0.    0.    0.    0.    0.   ]
 [0.227 0.159 0.227 0.159 0.227 0.    0.    0.    0.    0.   ]
 [0.134 0.191 0.134 0.134 0.134 0.272 0.    0.    0.    0.   ]
 [0.164 0.115 0.164 0.115 0.164 0.115 0.164 0.    0.    0.   ]
 [0.095 0.194 0.095 0.095 0.095 0.136 0.095 0.194 0.    0.   ]
 [0.128 0.09  0.128 0.09  0.128 0.09  0.128 0.09  0.128 0.   ]
 [0.096 0.096 0.096 0.096 0.096 0.096 0.096 0.096 0.096 0.137]]

position 0 attends only to itself: True
every row has zero weight on the future: True
every row still sums to 1: True
  row 0: 1 non-zero weight(s) -> positions 0..0
  row 1: 2 non-zero weight(s) -> positions 0..1
  row 2: 3 non-zero weight(s) -> positions 0..2
  row 9: 10 non-zero weight(s) -> positions 0..9

naive 'zero it out afterwards' row sums: [0.117 0.243 0.317 0.424 0.517 0.627 0.717 0.84  0.9
-> not 1. Masking before the softmax is what keeps it a distribution.
```



Left: the encoder sees everything. Right: the same sentence with the causal mask — strictly lower-triangular, so each token sees only itself and its past.

The verification

ROW	NON-ZERO WEIGHTS	CAN SEE
0	1	position 0 only — weight exactly 1.000
1	2	positions 0–1
2	3	positions 0–2
9	10	positions 0–9 (the whole sentence)

WHY -INF AND NOT A LARGE NEGATIVE NUMBER

Because $e^{-\infty}$ is exactly 0, so the masked weight is exactly zero — no leakage at any precision. A merely large negative number like −1e9 leaves a residue that is tiny but nonzero, and at generation time a model that can see 1e−9 of the future is a model that can cheat. NumPy handles this cleanly: `-inf` minus the row max is still `-inf`, and `np.exp(-inf)` is `0.0` with no warning.

One caveat: if an entire row were masked, every score would be `-inf`, and the row's sum would be 0 — giving `nan`. The `k=1` on line 12 is what prevents this, by always leaving the diagonal visible.

17

Final Exercise 3 — More heads, smaller dimension

The task: rerun with `num_heads=8` and `d_model=8`, so `d_head=1`. What breaks, and why does `d_model % num_heads == 0` matter?

THE SHORT ANSWER

Nothing breaks. `8 % 8 == 0`, so the assert passes and the code runs to completion with the correct output shape. That is the trap in the question. What degrades is each head's *expressiveness*, and the thing that actually breaks is a head count that does not divide the model width.

Solution

```python
# ============================================================================
# Section 8, Exercise 3 - num_heads=8 with d_model=8, so d_head=1
# ============================================================================

out8, w8 = multi_head_attention(X, num_heads=8, d_model=8, seed=1)
print("num_heads=8, d_model=8 -> runs fine. output shape:", out8.shape)
print("d_head =", 8 // 8)

# Nothing raises. The assert passes because 8 % 8 == 0. What degrades is the
# QUALITY of each head: with d_head=1, Qh and Kh are (seq_len, 1) columns, so
#     scores = Qh @ Kh.T
# is an OUTER PRODUCT of two vectors - a rank-1 matrix. A head can only express
# "how strongly does each token emit" x "how strongly does each token receive".
# It cannot represent a genuine pairwise relationship at all.
print("\nrank of one head's score matrix:")
for nh in (8, 4, 2):
    d_head = 8 // nh
    rng = np.random.RandomState(1)
    W_q = rng.randn(X.shape[1], d_head) * 0.3
    W_k = rng.randn(X.shape[1], d_head) * 0.3
    scores = (X @ W_q) @ (X @ W_k).T
    print(f"  num_heads={nh}, d_head={d_head}: rank {np.linalg.matrix_rank(scores)}"
          f"  (upper bound is d_head = {d_head})")

# There is a second, quieter problem: the scaling divisor is sqrt(d_head)=1,
# so a d_head=1 head gets no variance control at all.
print("\nsqrt(d_head) for d_head=1:", np.sqrt(1), "-> the scaling becomes a no-op")

# --- What ACTUALLY breaks: divisibility -------------------------------------
try:
    multi_head_attention(X, num_heads=3, d_model=8, seed=1)
except AssertionError as e:
    print("\nnum_heads=3, d_model=8 ->  AssertionError:", e)

print("""
Why d_model % num_heads == 0 matters: each head produces a (seq_len, d_head)
block, and the heads are CONCATENATED back into (seq_len, num_heads * d_head).
For that to be a drop-in replacement for the input - which is what residual
connections and block stacking require - the concatenated width has to land
back on exactly d_model. If d_model is not divisible by num_heads, integer
division silently loses columns and the shapes stop lining up.

With d_model=8 and num_heads=3, d_head = 8 // 3 = 2, so the heads concatenate
to width 6, not 8. The assert turns that silent shape bug into a loud error.""")
```

L4–6	Run it. Output shape `(10, 8)`, exactly as with 4 heads.
L8–14	The real consequence. With `d_head=1`, `Qh` and `Kh` are single columns, so `Qh @ Kh.T` is an **outer product** — a rank-1 matrix.
L15–22	Measure it directly across three configurations. The rank of a head's score matrix is bounded by its `d_head`. PYTHON `np.linalg.matrix_rank(scores)` computes the numerical rank. Rebuilding the projections with the same seed reproduces exactly what happens inside `multi_head_attention`.
L24–25	A second, quieter problem: the scaling divisor is $\sqrt{1} = 1$, so a one-dimensional head gets no variance control at all.
L28–31	**What actually breaks.** `num_heads=3` does not divide 8, and the assert fires. PYTHON `try / except AssertionError as e` catches the error so the notebook keeps running, and prints the message rather than stopping with a traceback.
L33–44	Why divisibility is structural rather than stylistic.

OUTPUT

```
num_heads=8, d_model=8 -> runs fine. output shape: (10, 8)
d_head = 1

rank of one head's score matrix:
  num_heads=8, d_head=1: rank 1  (upper bound is d_head = 1)
  num_heads=4, d_head=2: rank 2  (upper bound is d_head = 2)
  num_heads=2, d_head=4: rank 4  (upper bound is d_head = 4)

sqrt(d_head) for d_head=1: 1.0 -> the scaling becomes a no-op

num_heads=3, d_model=8 ->  AssertionError: d_model must be divisible by num_heads

Why d_model % num_heads == 0 matters: each head produces a (seq_len, d_head)
block, and the heads are CONCATENATED back into (seq_len, num_heads * d_head).
For that to be a drop-in replacement for the input - which is what residual
connections and block stacking require - the concatenated width has to land
back on exactly d_model. If d_model is not divisible by num_heads, integer
division silently loses columns and the shapes stop lining up.

With d_model=8 and num_heads=3, d_head = 8 // 3 = 2, so the heads concatenate
to width 6, not 8. The assert turns that silent shape bug into a loud error.
```

WHY RANK 1 IS A REAL LIMITATION

With $d_{\text{head}} = 1$, each head's scores factor as an outer product:

$$S = qk^T, \qquad S_{ij} = q_i k_j$$

Every entry is a product of a per-query number and a per-key number. So the head can only express "how strongly does token i emit" times "how strongly does token j receive". It **cannot** represent a genuinely pairwise relationship such as "*this* pronoun goes with *that* noun, but not with the other one" — that would need rank 2 or more.

The measured ranks confirm the bound exactly: `d_head=1` gives rank 1, `d_head=2` gives rank 2, `d_head=4` gives rank 4. This is why real models keep `d_head` at 64–128 and add heads by *widening the model*, never by slicing a fixed width thinner.

Why the divisibility assert exists

Each head yields a `(n, d_head)` block, and the blocks are concatenated to `(n, num_heads × d_head)`. For the result to be a drop-in replacement for the input — which residual connections and block stacking both require — that width must land back on exactly `d_model`.

$$H \cdot d_{\text{head}} = H \cdot \lfloor d_{\text{model}} / H \rfloor \;=\; d_{\text{model}} \quad \text{only when} \quad H \mid d_{\text{model}}$$

With `d_model=8` and `num_heads=3`, integer division gives `d_head = 2`, so the heads concatenate to width 6 rather than 8 — and `W_o` would then produce the wrong output width. The assert converts a silent shape bug into a loud, immediate error.

18 Final Exercise 4 — Stack encoder blocks

The task: modify `encoder_block` so it can be called twice, feeding the first output into the second. Confirm the shapes still match at every step.

> **THE NAIVE VERSION RUNS, AND IS WRONG**
>
> Calling `encoder_block(encoder_block(X))` produces the right shapes and no error. But look at the first line of the block's body: it **adds positional encoding**. Calling it twice adds the positional signal twice, so by layer 2 the tokens carry `embedding + 2×PE`. In a real transformer, position is added exactly once, before block 1. No exception, no shape mismatch — just quietly wrong numbers.

Solution

```python
# ============================================================================
# Section 8, Exercise 4 - Stack encoder blocks (N = 2)
# ============================================================================
# The naive way is to call encoder_block twice. It runs, and the shapes match.
# But look at what encoder_block does on line 1 of its body: it ADDS POSITIONAL
# ENCODING. Calling it twice adds the positional signal twice.

naive_1 = encoder_block(X, num_heads=4, seed=2)
naive_2 = encoder_block(naive_1, num_heads=4, seed=2)
print("naive stacking - shapes are fine:", X.shape, "->", naive_1.shape, "->", naive

# In a real transformer, positional encoding is added ONCE, before block 1.
# Every block after that just transforms whatever it is given. So split the
# block into "add position" and "the block itself":

def encoder_block_core(x, num_heads=4, seed=0):
    """One encoder block WITHOUT re-adding positional encoding."""
    attn_out, _ = multi_head_attention(x, num_heads=num_heads,
                                       d_model=x.shape[-1], seed=seed)
    x = layer_norm(x + attn_out)              # residual + norm
    ff_out = feed_forward(x, seed=seed + 1)
    x = layer_norm(x + ff_out)                # residual + norm
    return x

def encoder_stack(token_embeddings, N=2, num_heads=4, seed=0):
    seq_len, d_model = token_embeddings.shape
    x = token_embeddings + positional_encoding(seq_len, d_model)   # once, up front
    print(f"  after +PE            : {x.shape}")
    for n in range(N):
        x = encoder_block_core(x, num_heads=num_heads, seed=seed + n)
        print(f"  after block {n + 1}        : {x.shape}")
    return x

print("\ncorrect stacking, N=2:")
stacked = encoder_stack(X, N=2, num_heads=4, seed=2)
print("input", X.shape, "-> output", stacked.shape,
      "  shapes match:", stacked.shape == X.shape)

# How different are the two approaches?
diff = np.abs(naive_2 - stacked).max()
print(f"\nmax elementwise difference, naive vs correct: {diff:.4f}")
print("-> the shapes agree, so nothing errors. The VALUES are wrong, silently.")

# Shape preservation is what makes depth possible at all. It holds for any N:
print()
```

```python
for N in (1, 2, 4, 8):
    x = X + positional_encoding(*X.shape)
    for n in range(N):
        x = encoder_block_core(x, num_heads=4, seed=2 + n)
    print(f"  N={N}: output {x.shape}, all finite: {np.isfinite(x).all()}")

print("""
Why every block has to preserve shape: the residual connection computes
x + sublayer(x), which is only defined if sublayer(x) has exactly x's shape.
That constraint is what forces
    * multi-head attention to project back to d_model, and
    * the feed-forward network to go d_model -> d_ff -> d_model.
Get it right once and the block becomes a LEGO brick you can stack N times.
GPT-3 is this same brick, 96 times, with d_model=12288.""")

# One caveat worth seeing: this toy stack uses a fresh random projection per
# block and has no trained weights, so depth here does not make the output
# "better" - it just shows the plumbing is sound.
print("\nrepresentation drift per layer (mean |cos| between token pairs):")
x = X + positional_encoding(*X.shape)
for n in range(4):
    x = encoder_block_core(x, num_heads=4, seed=2 + n)
    norm = x / np.linalg.norm(x, axis=1, keepdims=True)
    sim = norm @ norm.T
    off = sim[~np.eye(len(x), dtype=bool)]
    print(f"  after block {n + 1}: {np.abs(off).mean():.3f}")
print("""
The tokens get steadily MORE alike with depth (0.58 -> 0.91). That is
'over-smoothing', or attention rank collapse: repeated averaging pulls every
token toward the same vector. Here it happens fast because the projections are
random and untrained. Trained transformers fight it with what is already in
this block - residual connections and layer norm - plus careful initialisation,
which is precisely why those pieces are not optional decoration.""")
```

L7–9

The naive stack, shown first so the failure is concrete rather than hypothetical. The shapes are perfect.

L15–23

The fix: split the block into “add position” and “the block itself”. `encoder_block_core` is the original minus the positional line.

> PYTHON
>
> `x.shape[-1]` reads `d_model` off the input so the function works at any width.

L25–32

The stack: add positional encoding once, then loop the core block N times. Printing the shape at each step is what the exercise asks you to confirm.

> PYTHON
>
> `for n in range(N)` runs the body N times, reassigning `x` each pass so each block consumes the previous one's output.

L34–37

Run it at N=2 and confirm input and output shapes match.

L39–42

Quantify the bug: the two approaches differ by up to 1.19 per element while agreeing perfectly on shape.

> PYTHON
>
> `np.abs(a - b).max()` gives the largest single-element discrepancy — a blunt but useful way to ask “are these the same array?”

L44–51

Confirm shape preservation holds for N = 1, 2, 4 and 8, and that nothing has drifted to `inf` or `nan`.

> PYTHON
>
> `np.isfinite(x).all()` is a cheap sanity check for numerical blow-up — worth doing whenever you iterate a transformation many times.

L53–62

Why shape preservation is forced rather than chosen.

L64–77

A closing measurement: how similar do token representations become as depth increases?

> PYTHON
>
> Dividing each row by its norm makes every vector unit length, so `norm @ norm.T` is the matrix of cosine similarities. `~np.eye(n, dtype=bool)` is a boolean mask selecting the off-diagonal entries — the diagonal is always 1 and would skew the average.

OUTPUT

```
naive stacking - shapes are fine: (10, 8) -> (10, 8) -> (10, 8)

correct stacking, N=2:
  after +PE            : (10, 8)
  after block 1        : (10, 8)
  after block 2        : (10, 8)
input (10, 8) -> output (10, 8)   shapes match: True

max elementwise difference, naive vs correct: 1.1913
-> the shapes agree, so nothing errors. The VALUES are wrong, silently.

  N=1: output (10, 8), all finite: True
  N=2: output (10, 8), all finite: True
  N=4: output (10, 8), all finite: True
  N=8: output (10, 8), all finite: True

Why every block has to preserve shape: the residual connection computes
x + sublayer(x), which is only defined if sublayer(x) has exactly x's shape.
That constraint is what forces
    * multi-head attention to project back to d_model, and
    * the feed-forward network to go d_model -> d_ff -> d_model.
Get it right once and the block becomes a LEGO brick you can stack N times.
GPT-3 is this same brick, 96 times, with d_model=12288.

representation drift per layer (mean |cos| between token pairs):
  after block 1: 0.582
  after block 2: 0.600
  after block 3: 0.871
  after block 4: 0.907

The tokens get steadily MORE alike with depth (0.58 -> 0.91). That is
'over-smoothing', or attention rank collapse: repeated averaging pulls every
token toward the same vector. Here it happens fast because the projections are
random and untrained. Trained transformers fight it with what is already in
this block - residual connections and layer norm - plus careful initialisation,
which is precisely why those pieces are not optional decoration.
```

The shapes, at every step

STAGE	SHAPE
Input embeddings	`(10, 8)`
After `+ PE`	`(10, 8)`
After block 1	`(10, 8)`
After block 2	`(10, 8)`
N = 8 blocks	`(10, 8)`, all finite

WHY EVERY SUBLAYER MUST PRESERVE SHAPE

The residual connection computes

$$x \leftarrow \mathrm{LayerNorm}(x + \mathrm{Sublayer}(x))$$

and $x + \mathrm{Sublayer}(x)$ is only defined when the two have identical shapes. That single requirement is what forces multi-head attention to project back to d_{model} via W^O, and the feed-forward network to go $d_{\text{model}} \to d_{ff} \to d_{\text{model}}$. Satisfy it once and the block becomes a component you can repeat indefinitely.

GPT-3 is this same brick 96 times over, with $d_{\text{model}} = 12288$ and $H = 96$.

One honest caveat

The last measurement shows token representations becoming steadily *more alike* with depth — mean off-diagonal cosine similarity rising 0.58 → 0.60 → 0.87 → 0.91. That is **over-smoothing**, or attention rank collapse: repeated averaging pulls every token toward the same vector.

WHY THAT HAPPENS HERE AND NOT IN GPT-3

These projections are random and untrained, so each block is close to a pure averaging operation — and averaging repeatedly converges to a constant. Trained transformers resist it using machinery already present in this block: residual connections preserve a copy of the original signal, layer norm keeps scales from collapsing, and the feed-forward nonlinearity pushes representations apart again. Careful initialisation matters too.

> Which is the real lesson of this exercise: residuals and layer norm are not decoration around the interesting part. Without them, depth actively destroys information.

19 Where this lands

Every piece in this notebook solves one specific problem. You can now name each one and say what fails without it.

PIECE	THE PROBLEM IT SOLVES	WHAT BREAKS WITHOUT IT
`Q @ K.T`	Measure which tokens are relevant to each other	No notion of relevance at all
`/ sqrt(d_k)`	Hold score variance at 1 as the model widens	Softmax saturates; gradients vanish (§15)
`softmax`	Turn scores into weights summing to 1	Outputs unbounded; no probabilistic reading
`x - max(x)`	Numerical stability	`nan` for large scores (§04)
Multiple heads	Several relationship types at once	One similarity structure only (§17)
`W_o`	Return concatenated heads to `d_model`	Heads never interact; residual breaks
Positional encoding	Give the model word order	“Dog bites man” = “man bites dog” (§08)
Residual `x + f(x)`	A path for gradients around each sublayer	Deep stacks do not train
Layer norm	Keep activations at a stable scale	Rank collapse with depth (§18)
Feed-forward	Per-token nonlinear computation	Attention only mixes; nothing is computed
Causal mask	Stop a decoder seeing the future	The model cheats at generation (§16)

The one-sentence version

Attention is a soft, differentiable dictionary lookup — every token writes a query, every token advertises a key, and the answer is a weighted average of values — and every other piece in the block exists to make stacking that operation ninety-six times deep actually trainable.

Three things the notebook exposed that the text does not say

1. **The demo skeleton has a bug.** The custom-sentence cell says `Q, K, V = X, X, X`, silently reusing the previous sentence. It must be `X_demo`. (§07)
2. **Final Exercise 1's suggested method cannot show its own effect.** Zero-padding leaves `Q @ K.T` untouched, so scaling gets flatter, not peakier. The variance argument needs vectors with signal in every dimension. (§15)
3. **Final Exercise 3's premise is wrong in an instructive way.** `num_heads=8` with `d_model=8` does not break — it runs perfectly and degrades silently to rank-1 attention. (§17)

Where to go next

1. **Add the learned projections.** Replace `Q = K = V = X` with `X @ W_q`, `X @ W_k`, `X @ W_v`. The mechanism does not change — only where the features come from. That is the whole difference between this notebook and a real attention layer.
2. **Cross-attention.** Let Q come from one sequence and K, V from another. That single change turns an encoder block into a decoder block, and is how translation worked before decoder-only models took over.
3. **Backpropagate through it.** Everything here is differentiable. Implementing the backward pass for `softmax` yourself is the fastest way to make the $p(1-p)$ argument from §15 stop being a formula and start being obvious.

Every code block on this page comes from a notebook executed end to end in one NumPy 2.4 kernel, and every output and figure is the real result of that run. The mathematics is rendered as inline SVG rather than by a script, so it stays sharp at any zoom and readable in both light and dark. To save a copy, use your browser's Print

command — the print stylesheet drops the sidebar and keeps equations, code and figures from splitting across pages.